

Aula 04

DataPrev (Perfil 3: Desenvolvimento de Software) Desenvolvimento de Software - 2026 (Pós-Edital)

Autor:
Vinicius Borges

09 de Julho de 2026

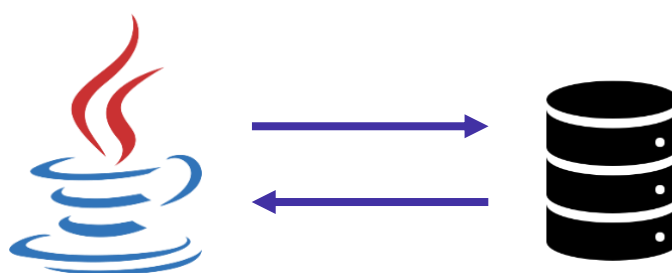
Índice

1) JPA - Teoria	3
2) JPA - Questões Comentadas	30
3) JPA - Lista de Questões	44
4) Hibernate - Teoria	55
5) Hibernate - Questões Comentadas	81
6) Hibernate - Lista de Questões	97

JAVA PERSISTENCE API (JPA)

Conceitos Gerais

JPA, ou **Java Persistence API**, é um framework baseado em POJOS (Plain Old Java Objects), destinada à **persistir objetos em Java**. Ela define um conjunto de regras e interfaces que **facilitam o mapeamento objeto-relacional** (ORM - Object-Relational Mapping), permitindo que desenvolvedores armazenem e recuperem objetos Java diretamente de um banco de dados relacional. Com isso, a JPA elimina a necessidade de o desenvolvedor escrever SQL complexo para operações de CRUD (*Create, Read, Update, Delete*), automatizando o processo de comunicação entre a aplicação Java e o banco de dados.



A **persistência de dados** é fundamental em sistemas corporativos, que geralmente trabalham com grande volume de dados e necessitam de uma camada de acesso eficiente e confiável. A JPA oferece uma **abstração que oculta as complexidades de comunicação com o banco de dados**, como a necessidade de conexões diretas e o uso de instruções SQL. Em vez disso, o desenvolvedor trabalha com uma camada de objetos Java, que são traduzidos em tabelas e colunas do banco de dados relacional. Essa abstração é especialmente útil em aplicações onde os modelos de dados tendem a ser complexos e ricos em relacionamentos.

Em termos práticos, a JPA funciona utilizando a técnica de ORM, onde **as classes Java são mapeadas para tabelas** no banco de dados e **os atributos das classes são mapeados para as colunas** dessas tabelas. Esse mapeamento é configurado por meio de anotações Java ou arquivos XML, permitindo que o desenvolvedor defina a estrutura e os relacionamentos entre os objetos de forma declarativa. Assim, quando uma aplicação utiliza JPA, ela pode "persistir" um objeto em um banco de dados com um simples comando, como `entityManager.persist()`, ao invés de ter que construir uma instrução SQL de inserção. Esse modelo simplifica o código e reduz a probabilidade de erros, ao mesmo tempo em que torna o código mais legível e fácil de manter.

Aqui é importante destacar que o JPA é uma **especificação oficial** – isso é, não possui código que possa ser executado. Ela é uma interface que necessita que alguém a implemente. Apesar disso, a especificação pode possuir classes, interfaces e anotações para auxiliar no desenvolvimento. Para implementarmos, usamos outras ferramentas, como o OpenJPA, Hibernate ou EclipseLink.

Mapeamento Objeto-Relacional (ORM)

O **mapeamento objeto-relacional** (ORM - Object-Relational Mapping) é uma técnica de desenvolvimento que permite a representação de dados armazenados em um banco relacional como objetos dentro de uma aplicação orientada a objetos, como uma aplicação Java. A ideia central do ORM é **eliminar a lacuna entre o mundo dos bancos de dados relacionais e o paradigma da orientação a objetos**, facilitando a persistência dos dados, ou seja, o armazenamento e recuperação de dados em um banco relacional a partir de objetos Java.

Em uma aplicação que utiliza ORM, as **classes Java representam tabelas** do banco de dados, os **atributos dessas classes correspondem às colunas** das tabelas e os **objetos representam os registros da tabela**. Isso significa que, ao invés de construir instruções SQL manualmente para cada operação de leitura, inserção, atualização ou exclusão, podemos utilizar as anotações e recursos do ORM para que esses objetos Java (entidades) sejam automaticamente sincronizados com o banco de dados.

CLASSES → TABELAS
ATRIBUTOS → COLUNAS
OBJETO → REGISTRO

O ORM possibilita o mapeamento entre as classes e tabelas e seus respectivos atributos e colunas, criando uma camada de abstração. Dessa forma, o desenvolvedor pode manipular dados no banco como se fossem objetos Java, sem precisar conhecer a estrutura SQL subjacente. A tradução entre o mundo dos objetos e o banco de dados é realizada pela implementação da JPA (Hibernate, por exemplo), que transforma essas entidades em instruções SQL e mantém o controle sobre o ciclo de vida dos objetos.

Essa camada de abstração facilita o trabalho com bancos de dados complexos, permite a reutilização do código e proporciona uma manutenção mais simples, pois o código Java passa a ser o ponto central de definição das regras de negócio, enquanto o framework gerencia a persistência.

Para mapear essas classes e atributos do Java para uma estrutura relacional relacionada, utilizamos **anotações específicas**. Em Java, uma **anotação é um recurso que permite adicionar metadados ao código**. As anotações, **identificadas pelo símbolo @** seguido do nome da anotação, não alteram diretamente a lógica de execução, mas fornecem informações adicionais que podem ser utilizadas pelo compilador, ferramentas de desenvolvimento ou frameworks para realizar ações específicas ou configurar comportamentos de maneira automática.

No contexto da JPA, as anotações são fundamentais para o mapeamento objeto-relacional (ORM), permitindo configurar as classes e atributos para se relacionarem diretamente com o banco de dados. As **principais anotações do JPA** são:

Anotação	Descrição
@Entity	Marca uma classe como uma entidade JPA, indicando que ela representa uma tabela no banco de dados. Para que uma classe seja persistida pelo JPA, é essencial que ela esteja anotada com @Entity.
@Table	Especifica a tabela do banco de dados à qual a entidade será mapeada. Sem essa anotação, a JPA considera que o nome da tabela será o mesmo da classe. Exemplo de uso: @Table(name = "usuarios").
@Id	Define o atributo que atuará como chave primária na tabela do banco de dados. É necessário que cada entidade tenha um atributo anotado com @Id para que o JPA possa identificar cada instância de forma única.
@GeneratedValue	Configura a geração automática de valores para a chave primária. A anotação pode especificar a estratégia de geração, como AUTO, IDENTITY, SEQUENCE ou TABLE, dependendo das regras de geração do banco de dados.
@Column	Permite especificar detalhes da coluna no banco de dados, como o nome, tipo de dado, se aceita valores nulos, e se é única. Não é obrigatória para cada campo, mas é útil quando queremos personalizar o mapeamento.
@OneToOne	Representa um relacionamento um-para-um.
@OneToMany	Representa um relacionamento um-para-muitos.
@ManyToOne	Representa um relacionamento muitos-para-um.
@ManyToMany	Representa um relacionamento muitos-para-muitos.
@JoinColumn	Define a coluna de junção para relacionamentos de entidades. Por exemplo, em um relacionamento @OneToOne, @JoinColumn especifica a coluna que atua como chave estrangeira, ligando as entidades.
@JoinTable	Em um relacionamento @ManyToMany, cria uma tabela intermediária (entidade associativa) para representar a relação entre as duas tabelas principais. Essa anotação especifica o nome da tabela de junção e as colunas de chave estrangeira que estabelecem o vínculo.

@Transient	Indica que um campo ou propriedade da entidade não deve ser persistido no banco de dados. Esse campo só existirá no contexto da aplicação, sem criar uma coluna correspondente.
@Embedded e @Embeddable	Permitem incluir uma classe dentro de uma entidade como um tipo composto (chave, por exemplo). @Embedded é usada para definir a instância do tipo composto, enquanto @Embeddable marca a classe que será embutida.
@NamedQuery e @NamedQueries	São utilizadas para definir consultas JPQL pré-definidas, que podem ser reutilizadas ao longo do código. @NamedQuery cria uma única consulta nomeada, enquanto @NamedQueries permite definir múltiplas consultas.
@Version	Anota um campo como um indicador de versão para controle de concorrência otimista. Isso é útil para gerenciar conflitos de atualização em aplicações onde múltiplas transações podem alterar a mesma entidade simultaneamente.

As anotações são um importante elemento do JPA e falaremos mais de algumas dessas anotações quando oportuno.

(FURB/Pref. Blumenau/2022) Sobre o JPA, analise as afirmativas:

I - JPA é um framework leve, baseado em POJOS (Plain Old Java Objects) para persistir objetos Java.

II - A Java Persistence API, diferente do que muitos imaginam, não é apenas um framework para Mapeamento Objeto-Relacional (ORM - Object-Relational Mapping), ela também oferece diversas funcionalidades essenciais em qualquer aplicação corporativa.

III - JPA provê limitadas funcionalidades para os programadores e também não representa uma simplificação do modelo de programação de persistência. A especificação JPA remove explicitamente o mapeamento relacional de objetos, em vez de depender das implementações de mapeamento específicas do fornecedor.

Estão CORRETAS as afirmativas:

- a) III, apenas.
- b) I, apenas.
- c) I e II, apenas.
- d) I, II e III.
- e) II, apenas.

Comentários:

Vamos analisar os itens.

I – Certo. O JPA é baseado em POJOS e seu objetivo é, de fato, persistir objetos Java.

II – Certo. O JPA oferece vários recursos que vão além de um simples ORM, com recursos como cache, gerenciamento de ciclo de vida, etc.

III – Errado. Fazer persistência de forma “tradicional”, com CRUD, é muito complexo – o JPA acaba sendo uma forma bem simplificada de se fazer isso. Além disso, não há remoção do mapeamento relacional.

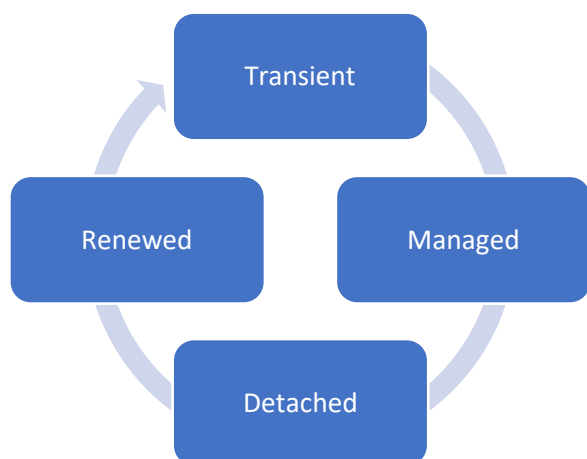
Ficamos com I e II como corretas. (Gabarito: Letra C)

Gerenciamento de Entidades

No JPA, o **gerenciamento de entidades** é realizado principalmente pelo **EntityManager**, que controla o ciclo de vida das entidades, realiza operações de persistência e sincroniza as alterações feitas nas entidades com o banco de dados. O **EntityManager** é o ponto central de interação com o banco de dados, permitindo que operações de CRUD sejam realizadas de maneira eficiente e segura.

Esse gerenciador atua como uma interface entre a aplicação e o banco de dados, facilitando o **gerenciamento das entidades em diferentes estados do ciclo de vida** – assunto que veremos logo em seguida. Ele é responsável por manter o controle das entidades no contexto de persistência, aplicando as alterações ao banco de dados de acordo com a configuração das transações.

Bom, cada entidade tem um **ciclo de vida**. Esse ciclo define o conjunto de estados pelos quais uma entidade passa desde sua criação até sua eventual remoção do banco de dados. Vamos focar nos principais estados que uma entidade passa durante seu ciclo: **Transient**, **Managed**, **Detached** e **Renewed**.



O estado **TRANSIENT** ocorre quando uma **entidade é criada, mas ainda não foi associada a um contexto de persistência**. Nesse estado, a entidade só existe em memória e não está vinculada ao banco de dados. Nesse caso, ela não é monitorada pelo gerenciador e não há sincronização com o banco de dados. Para persistir, usamos o método **persist()** do **EntityManager**.

No estado **MANAGED** (gerenciado), **associamos uma entidade a um contexto de persistência**, e ela passa a ser monitorada pelo **EntityManager**. Assim, qualquer alteração feita nela é automaticamente propagada ao

banco de dados no fim da transação.

Como devemos atender aos princípios ACID de transações, principalmente no que tange os aspectos da atomicidade, a sincronização automática só ocorrerá a partir de um método **commit()**. Então, usamos o método **begin()** para iniciar o monitoramento e gerenciamento, e o método **commit()** para encerrar e confirmar as transações.

Java

```
entityManager.getTransaction().begin();  
entityManager.persist(pessoa); // `pessoa` agora está gerenciada.  
entityManager.getTransaction().commit();
```

O método `persist()` não é o único que permite transicionar entre os dois primeiros estados. Podemos usar métodos que tragam a entidade para o contexto de persistência, como o `find()` ou `getReference()`, além de reanexar uma entidade desanexada ao contexto de persistência com o método `merge()`.

O próximo estado é o `DETACHED`. Esse estado **ocorre quando uma entidade gerenciada é removida do contexto de persistência**, ou seja, o `EntityManager` para de monitorar. A entidade ainda existe em memória, mas as alterações feitas nela não serão aplicadas ao banco de dados – ou seja, alterações feitas em uma entidade `detached` não serão refletidas no banco a menos que seja reanexada ao contexto.

Para que uma entidade `detached` volte a ser gerenciada, podemos usar o método `merge()`. Ele **reanexa a entidade ao contexto de persistência**, criando uma nova instância gerenciada com os mesmos dados da entidade `detached` e sincronizando as mudanças com o banco de dados.

Java

```
Pessoa pessoa = entityManager.find(Pessoa.class, 1L); // Gerenciada  
entityManager.detach(pessoa); // Agora `pessoa` está detached  
pessoa.setNome("Novo Nome"); // Modificação não será refletida no banco  
entityManager.merge(pessoa); // Reanexa e sincroniza a mudança
```

O último estado é o `REMOVED`. Uma entidade gerenciada entra no estado `removed` quando é marcada para exclusão com o método `remove()` do `EntityManager`. Após a confirmação da transação, a entidade será removida fisicamente do banco de dados.

Vamos ver um resumo dos métodos associados ao `EntityManager`!



RESUMINDO

Método	Descrição
<code>persist(Object entity)</code>	Associa a entidade ao contexto de persistência, colocando-a no estado managed. Quando a transação é confirmada, a entidade é inserida no banco de dados.
<code>merge(Object entity)</code>	Reanexa uma entidade detached ao contexto de persistência, sincronizando-a com o banco. Retorna uma nova instância gerenciada com os dados da entidade original.
<code>remove(Object entity)</code>	Marca uma entidade gerenciada para exclusão. Após o commit da transação, a entidade será removida fisicamente do banco de dados.
<code>find(Class<T> entityClass, Object primaryKey)</code>	Busca uma entidade no banco de dados com base em sua chave primária. Se encontrada, a entidade é trazida para o estado managed no contexto de persistência.
<code>getReference(Class<T> entityClass, Object primaryKey)</code>	Retorna uma referência a uma entidade com a chave primária fornecida, sem carregar os dados completos. Útil para otimizar desempenho em cenários de carregamento sob demanda.
<code>flush()</code>	Sincroniza o estado atual do contexto de persistência com o banco de dados, aplicando as alterações pendentes sem fechar a transação.
<code>refresh(Object entity)</code>	Atualiza o estado de uma entidade gerenciada com os dados mais recentes do banco, descartando quaisquer alterações feitas localmente.
<code>detach(Object entity)</code>	Remove a entidade do contexto de persistência, passando-a para o estado detached. Alterações subsequentes na entidade não serão refletidas no banco.
<code>clear()</code>	Remove todas as entidades do contexto de persistência, desanexando-as e colocando-as no estado detached.

<code>close()</code>	Fecha o EntityManager, liberando recursos e desanexando todas as entidades. Após o fechamento, o EntityManager não pode ser reutilizado.
<code>lock(Object entity, LockModeType lockMode)</code>	Define um bloqueio para uma entidade gerenciada no banco de dados. Pode ser usado para gerenciar concorrência.
<code>createQuery(String qlString)</code>	Cria uma consulta JPQL (Java Persistence Query Language) com base na string fornecida.
<code>createNamedQuery(String name)</code>	Cria uma consulta JPQL com base em uma consulta nomeada previamente definida.
<code>getTransaction()</code>	Retorna o objeto EntityTransaction associado ao EntityManager, usado para controlar as transações de forma programática em ambientes sem JTA.
<code>find(Class<T> entityClass, Object primaryKey)</code>	Busca uma entidade no banco de dados com base em sua chave primária. Se encontrada, a entidade é trazida para o estado managed no contexto de persistência.

Bastante atenção nesses métodos, pois eles são cobrados!



(FCC/TRT 14/2022) Em uma aplicação Java que utiliza JPA 2.0, o método da interface *EntityManager*, utilizado em operações de alteração de dados, que executa um comando SQL UPDATE na tabela do banco de dados é o

- a) *persist*.
- b) *merge*.
- c) *executeUpdate*.

- d) save.
- e) pushUpdate.

Comentários:

Para atualizarmos os valores, numa operação equivalente ao SQL UPDATE, usamos o método merge(). (Gabarito: Letra B)

Gerenciamento de Chave Primária

No JPA, a definição de chaves primárias é fundamental para que cada entidade seja identificada de forma única no banco de dados. A **chave primária é usada como identificador exclusivo da entidade** e permite que o JPA realize operações como leitura, atualização e exclusão de maneira eficaz. A seguir, vamos explorar os principais aspectos relacionados à criação de chaves primárias, incluindo anotações, estratégias de geração de chaves e o uso de chaves compostas.

Para definir uma chave primária em uma entidade, utilizamos a anotação **@Id** no atributo que representará a chave primária. Essa anotação é obrigatória, pois indica ao JPA que o campo anotado será utilizado como o identificador único da entidade no banco de dados. Veja um exemplo simples:

```
Java

@Entity
public class Pessoa {
    @Id
    private Long id;
    private String nome;
}
```

Nesse exemplo, o campo **id** é marcado como a chave primária da entidade **Pessoa**. Isso significa que cada instância da classe **Pessoa** terá um valor exclusivo para **id**.

Além disso, é possível fazer um gerenciamento automático de chaves, facilitando ainda mais o desenvolvimento. Isso é feito a partir da anotação **@GeneratedValue**, que permite configurar estratégias de geração automática de IDs. O atributo **strategy** da anotação **@GeneratedValue** permite escolher o método de geração de chave primária a ser utilizado, conforme as opções da enumeração **GenerationType**:

- **GenerationType.AUTO**: Delega ao provedor JPA a escolha da estratégia mais apropriada, com base nas configurações do banco de dados.
- **GenerationType.IDENTITY**: Usa a geração de IDs autoincrementados do banco de dados, comum em SGBDs como MySQL.
- **GenerationType.SEQUENCE**: Requer o uso de sequências do banco de dados para gerar IDs únicos, suportado por bancos como Oracle e PostgreSQL.
- **GenerationType.TABLE**: Usa uma tabela especial no banco de dados para simular a geração de uma sequência, garantindo independência em relação ao SGBD.

Olha só um exemplo de implementação do tipo **IDENTITY**:

Java

```
@Entity
public class Pessoa {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;
}
```

(FCC/TRT 19/2022) Considere a classe abaixo em uma aplicação que utiliza JPA e Hibernate.

```
@Entity
public class Cliente {
    @Id
    @GeneratedValue(strategy = GenerationType. ...)
    private Long id;
    // atributos e métodos restantes
}
```

Para informar ao provedor de persistência que os valores a serem atribuídos ao identificador único (id) serão gerados pela coluna de autoincremento da tabela do banco de dados, utiliza-se, na lacuna I, o valor

- a) TABLE
- b) AUTO_GENERATED
- c) AUTO_INCREMENT
- d) IDENTITY
- e) SELF_GENERATED

Comentários:

Como queremos uma chave que irá ter autoincrementos, o valor a ser utilizado deve ser o IDENTITY. (Gabarito: Letra D)



(FGV/DPE RS/2023) Quando o J2EE foi substituído pelo JEE5, e as configurações via XML passaram a ser efetuadas com base em anotações, as entidades anotadas do JPA passaram a ser utilizadas no lugar dos Entity Beans.

Para definir uma chave primária com geração automática de valores, de forma independente do tipo de banco de dados, a configuração deve ser:

- (A) @Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
- (B) @Column(name="Id")
@Sequence(name="seq_chaves")
- (C) @Column(name="Id")
@Table(name="tab_chaves")
- (D) @Id
@GeneratedValue(strategy=GenerationType.SEQUENCE,
generator="sequenceGen")
@SequenceGenerator(name="sequenceGen",
sequenceName="seq_chaves")
- (E) @Id
@GeneratedValue(strategy = GenerationType.TABLE,
generator = "tableGen")
@TableGenerator(name = "tableGen", table = "tab_chaves",
pkColumnName = "id", pkColumnValue = "tabelaX",
valueColumnName = "valor")

Comentários:

A resposta correta para definir uma chave primária com geração automática de valores de forma independente do tipo de banco de dados é a letra E. Queremos uma anotação que irá apontar o uso de uma estratégia abordando uma tabela independente do banco de dados para a geração de chaves – que nos indica o uso de `strategy=GenerationType.TABLE` – dentro da anotação `@GeneratedValue`. A única alternativa que aborda essa anotação corretamente é a letra E. O

`generator = "tableGen"` permite especificar o nome da tabela geradora, colunas e outros detalhes acerca dela. (Gabarito: Letra E)

Chaves Compostas

Chaves compostas são aquelas compostas por mais de um atributo (coluna). Nesses casos, precisamos de um tratamento especial para o JPA ocorrer corretamente. Isso é feito por duas abordagens distintas, usando a anotação `@IdClass` ou com as anotações `@EmbeddedId` e `@Embeddable`.

A anotação `@IdClass` permite **especificar uma classe externa** que contém os campos que compõem a chave primária. A classe que representa a chave composta deve implementar `Serializable` e ter campos que correspondam aos campos da chave primária da entidade.

Java

```
@Entity
@IdClass(PessoaId.class)
public class Pessoa {
    @Id
    private String nome;
    @Id
    private String sobrenome;
    private String endereco;
}

public class PessoaId implements Serializable {
    private String nome;
    private String sobrenome;
    // Construtores, equals e hashCode são necessários
}
```

Nesse exemplo, a classe `PessoaId` define os campos `nome` e `sobrenome`, que juntos compõem a chave primária da entidade `Pessoa`.

A outra forma de definir uma chave composta é usando as anotações `@EmbeddedId` e `@Embeddable`. Com `@Embeddable`, criamos uma classe que encapsula os campos da chave composta, e com `@EmbeddedId`, definimos que essa classe será usada como chave primária. Veja outro exemplo.

Java

```
@Embeddable
public class PessoaId implements Serializable {
    private String nome;
    private String sobrenome;
    // Construtores, equals e hashCode são necessários
}

@Entity
public class Pessoa {
    @EmbeddedId
    private PessoaId id;
    private String endereco;
}
```



HORA DE
PRATICAR!

(FUNDATEC/PROCERGS/2023) O trecho de código abaixo, em Java Persistence API (JPA) versão 2.2, apresenta a declaração de uma chave primária composta de uma entidade de um banco de dados.

```
@_____
public class PessoaId implements Serializable {
    String nome;
    String sobrenome;
}

@_____
public class Pessoa {
    @_____ PessoaId id;
    String endereco;
}
```

Assinale a alternativa que preenche, correta e respectivamente, as lacunas do trecho de código acima.

(A) Embeddable – Entity – EmbeddedId

- (B) Instance – Entity – Primary
- (C) Association – Table – Primary
- (D) Instance – Table – EmbeddedId
- (E) Embeddable – Entity – Primary

Comentários:

Precisamos identificar as anotações para cada classe. No contexto do JPA, para definir uma chave primária composta, utilizamos uma classe que agrupa os campos que compõem a chave e a anotamos com `@Embeddable`. Isso indica que essa classe pode ser embutida como parte de uma outra entidade para compor uma chave primária – preenchendo nosso primeiro campo.

No segundo campo, temos a anotação `@Entity`, indicando que se trata de uma entidade JPA, ou seja, uma classe persistente, equivalente a uma tabela no banco de dados. Por fim, o terceiro campo recebe a anotação `@EmbeddedID`, que informa que esse campo é uma chave primária composta. (Gabarito: Letra A)

Gerenciamento de Relacionamentos

O gerenciamento de relacionamentos entre entidades é essencial para modelar a estrutura de dados de maneira que reflita as associações entre diferentes tabelas do banco de dados. Em aplicações corporativas, entidades raramente são isoladas, sendo comum que haja relacionamentos complexos como "um-para-muitos", "muitos-para-muitos", entre outros, conforme vimos anteriormente nas anotações.

Em alguns casos, é necessário que seja atribuída uma responsabilidade a um dos lados dos relacionamentos, e isso se dá a partir do `mappedBy`. Ele é usado em relacionamentos bidirecionais para indicar qual lado do relacionamento é o "dono". O "dono" do relacionamento é a entidade que controla a persistência e a chave estrangeira no banco de dados.

Quando usamos `mappedBy`, estamos indicando que a responsabilidade de gerenciar a relação está do outro lado do relacionamento, o que evita duplicação de informações e ajuda a JPA a entender como persistir e atualizar os dados corretamente.

A partir dessa definição do `mappedBy`, uma segunda configuração que podemos trabalhar é `cascata`, ou `cascade`. Essa propriedade controla como as **operações de persistência**, remoção, atualização, entre outras, são **propagadas entre as entidades relacionadas** – é basicamente o nosso `on delete cascade` do SQL. As opções de cascata incluem:

- **`CascadeType.PERSIST`**: Quando a entidade "dona" é persistida, as entidades relacionadas também são.

- **CascadeType.MERGE**: Quando a entidade "dona" é mesclada com outra entidade, as entidades relacionadas também são.
- **CascadeType.REMOVE**: Remove as entidades relacionadas ao excluir a entidade "dona".
- **CascadeType.ALL**: Aplica todos os tipos de cascata (persistir, mesclar, remover, etc.) entre a entidade "dona" e as entidades relacionadas.

Exemplificando:

Java

```
@OneToMany(mappedBy = "departamento", cascade = CascadeType.ALL)  
private List<Funcionario> funcionarios;
```

Por fim, a terceira configuração que iremos abordar aqui é a definição da **estratégia de carregamento**, ou **fetching**. Essa estratégia define quando as entidades relacionadas devem ser carregadas do banco de dados. Existem duas principais estratégias de carregamento:

- **EAGER** (Carregamento Antecipado): A entidade relacionada é carregada imediatamente, junto com a entidade principal. Isso pode ser útil em relacionamentos um-para-um, onde é esperado que as duas entidades sejam carregadas juntas.
- **LAZY** (Carregamento Sob Demanda): A entidade relacionada é carregada apenas quando é acessada pela primeira vez. Esse é o comportamento padrão para relacionamentos @OneToMany e @ManyToMany e é mais eficiente para bancos de dados, pois evita o carregamento desnecessário de dados.

Consultas JPQL

a **Java Persistence Query Language (JPQL)** é a **linguagem de consulta** usada para buscar, atualizar e manipular entidades armazenadas no banco de dados. Embora semelhante ao SQL, a JPQL é orientada a objetos, o que significa que trabalha diretamente com as classes e atributos definidos na aplicação Java, em vez de tabelas e colunas do banco de dados. Essa abordagem facilita a integração com o modelo de dados da aplicação, proporcionando uma abstração de persistência mais alinhada com o paradigma orientado a objetos. Temos muitas semelhanças com o SQL: cláusulas SELECT, FROM, WHERE, entre outras. Veja um exemplo de sintaxe:

```
SELECT e.name, e.salary FROM Employee e WHERE e.department.id = :deptId
```

Ainda, é possível que criemos as **Named Queries**, ou consultas nomeadas. Elas são definidas de forma estática na classe da entidade e permitem reutilizar consultas JPQL em diferentes partes da aplicação. Elas são especialmente úteis para consultas frequentes e otimizam o desempenho ao serem pré-compiladas – remetendo muito aos *stored procedures* do SQL.

Java

```
@Entity
@NamedQuery(
    name = "Employee.findByDepartment",
    query = "SELECT e FROM Employee e WHERE e.department.id = :deptId"
)
public class Employee { ... }
```

É possível também tornar as consultas mais dinâmicas. O JPQL oferece o uso de **parâmetros** que agem exatamente nessa área. Temos dois tipos de parâmetros:

- **Parâmetros Nomeados** (:nomeDoParametro): Utilizam o nome do parâmetro com o prefixo **:**. Esses parâmetros são definidos com o método `setParameter`. Exemplo:

Java

```
TypedQuery<Employee> query = entityManager.createQuery("SELECT e FROM Employee e WHERE e.name = :name", Employee.class);
query.setParameter("name", "Carlos");
```

- **Parâmetros Posicionais** (?posição): Utilizam uma posição numérica, **começando em 1**. Estes são menos comuns, pois sua legibilidade é inferior aos parâmetros nomeados.

Java

```
SELECT e FROM Employee e WHERE e.name = ?1
```

Query e TypedQuery

A **classe responsável** por representar e executar consultas é a **Query**. Ela fornece métodos para configurar e executar consultas JPQL ou SQL nativo, retornando resultados personalizados de acordo com a necessidade.

A classe **Query** é uma classe genérica que permite executar consultas JPQL e SQL nativas e manipular os resultados sem uma tipagem específica. Ela pode ser utilizada para retornar listas de entidades, realizar atualizações e exclusões em massa e até para consultas complexas.

Como uma classe, a Query é acompanhada de alguns métodos:

Método	Descrição
setParameter(String name, Object value)	Define um valor para um parâmetro na consulta.
getResultList()	Executa a consulta e retorna o resultado como uma lista de objetos. Esse método é ideal para consultas que retornam múltiplos resultados.
getSingleResult()	Executa a consulta e retorna um único resultado. Se não houver nenhum resultado ou se houver mais de um, lança uma exceção.
executeUpdate()	Usado para operações de atualização ou exclusão em massa, que modificam várias linhas de uma vez no banco de dados. Retorna o número de registros afetados pela operação.
setMaxResults(int maxResult)	Limita o número de resultados retornados pela consulta, útil para implementar paginação.
setFirstResult(int startPosition)	Define a posição inicial do primeiro resultado, útil para paginação em combinação com setMaxResults.

Além disso, o JPA oferece uma subclasse chamada **TypedQuery**, que permite consultas tipadas para resultados mais seguros e expressivos. Isso significa que o tipo de resultado esperado é especificado no momento da criação da consulta, permitindo maior segurança e evitando a necessidade de castings explícitos. Os métodos associados ao TypedQuery são similares aos da Query, mas com tipagem explícita.



HORA DE PRATICAR!

(FCC/TRT 12/2023) Em uma aplicação Java que utiliza JPA, em condições ideais, considere manager um objeto válido criado por meio da instrução abaixo.

```
EntityManager manager =  
Persistence.createEntityManagerFactory("trtPU").createEntityManager();
```

A instrução `manager.createNamedQuery("Acesso.findByUserSenha", Acesso.class)`; irá retornar um objeto do tipo

- a) `List<Acesso>`
- b) `Acesso`
- c) `Query<Acesso>`
- d) `TypedQuery<Acesso>`
- e) `ArrayList<Acesso>`

Comentários:

Sabemos que estamos fazendo uma consulta – mais especificamente, criando uma Named Query. Nos resta saber se a classe utilizada é `Query` ou `TypedQuery`. Para essa identificação, precisamos observar o uso do segundo parâmetro – `Acesso.class`. Como estamos definindo uma tipagem, podemos classificar o objeto de retorno como `TypedQuery<Acesso>`. (Gabarito: Letra D)

(FCC/TRT 5/2022) Considere o trecho de código a seguir, presente em um método de uma classe de aplicação criada utilizando-se JPA.

```
TypedQuery<Departamento> query = manager.createNamedQuery("Departamento.findAll",  
Departamento.class); List<Departamento> departamentos = ...!...;
```

`Departamento.findAll` é uma query nomeada na classe `Departamento`, que retorna todos os departamentos cadastrados na tabela `departamento` do banco de dados. O objeto `manager` é um objeto do tipo `EntityManager` criado sobre uma conexão válida com um banco de dados. Considere que todos os recursos necessários foram importados e que a conexão com o banco de dados foi realizada com sucesso. Nestas condições, para se obter a lista de departamentos retornada da tabela do banco de dados a lacuna `!` deve ser corretamente preenchida por

- a) `query.getResult().toList()`
- b) `manager.excuteQuery()`
- c) `query.getResult()`
- d) `query.getResultList()`
- e) `manager.executeQuery(query)`

Comentários:

O método `getResultList()` da classe `TypedQuery` é usado para executar uma consulta e retornar uma lista de resultados. No caso do código apresentado, a consulta nomeada "Departamento.findAll" é configurada para retornar todos os registros da entidade Departamento na tabela correspondente do banco de dados – sendo o `getResultList()` adequado nessa situação. (Gabarito: Letra D)

Implementação

Como vimos, o **JPA é uma especificação** – e, portanto, necessita de alguma aplicação externa para ser implementando. Ela somente descreve como o ORM e a persistência deve funcionar nas aplicações Java, restando a ferramentas que forneçam a implementação, como o Hibernate, EclipseLink e OpenJPA.

Além de escolher uma implementação, uma aplicação que utiliza JPA também exige **configurações específicas**, como a definição do banco de dados, as propriedades de conexão e o comportamento de gerenciamento de entidades. Essas configurações geralmente são definidas no arquivo **persistence.xml**, que é o ponto de configuração principal para um projeto JPA.

Quanto às implementações, temos:

- **Hibernate:** é a implementação de JPA mais popular e amplamente usada, conhecida por sua estabilidade e grande conjunto de funcionalidades. Ele fornece extensões além da especificação JPA, como caching de segundo nível, suporte a consultas SQL nativas mais robusto e um conjunto avançado de estratégias de mapeamento.
- **EclipseLink:** é a implementação de referência oficial para JPA, desenvolvida e mantida pela Eclipse Foundation. Suporta funcionalidades avançadas, como mapeamentos complexos de objetos para XML, caching avançado e integração com o padrão OSGi.
- **OpenJPA:** é um projeto da Apache e também é uma implementação bastante usada em alguns ambientes corporativos. É conhecida por sua boa performance e escalabilidade em cenários de carga pesada, mas possui menos funcionalidades extras do que o Hibernate.

Para a integração, precisamos configurar corretamente o arquivo **persistence.xml**. O arquivo **persistence.xml** é onde as configurações do JPA são definidas. Ele geralmente fica localizado em META-INF e contém as configurações essenciais para o provedor de persistência, o banco de dados e outras propriedades específicas do JPA.

Temos algumas configurações básicas que você precisa saber:

- **persistence-unit:** O elemento **persistence-unit** define uma unidade de persistência, que agrupa as configurações relacionadas ao provedor JPA e ao banco de dados. O atributo **name** identifica a unidade de persistência, e o **transaction-type** pode ser **RESOURCE_LOCAL** (para controle manual de transações) ou **JTA** (para controle de transações gerenciadas por um container).
- **Provedor JPA (<provider>):** O elemento **<provider>** especifica o provedor JPA a ser usado. Ele deve apontar para a classe de implementação do provedor. Exemplo de configuração para o Hibernate:

```
<provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
```

- **Configuração de Conexão com o Banco de Dados:** As propriedades `javax.persistence.jdbc.driver`, `javax.persistence.jdbc.url`, `javax.persistence.jdbc.user`, e `javax.persistence.jdbc.password` são usadas para configurar a conexão com o banco de dados.
- **Dialetos e Esquemas:** Muitos provedores JPA, como o Hibernate, exigem a especificação de um dialeto SQL (`hibernate.dialect`) que é específico para o tipo de banco de dados, como `MySQL8Dialect` ou `PostgreSQLDialect`. A propriedade `hibernate.hbm2ddl.auto` define como o Hibernate deve tratar o esquema do banco de dados, com valores como:
 - `validate`: Valida o esquema em relação às entidades mapeadas.
 - `update`: Atualiza o esquema conforme as entidades.
 - `create`: Cria um novo esquema a cada inicialização.
 - `create-drop`: Cria um novo esquema e o apaga ao encerrar a aplicação.
- **Configurações de Log e Debug:** Propriedades como `hibernate.show_sql` e `hibernate.format_sql` ajudam no desenvolvimento, mostrando as consultas SQL geradas e formatando-as para facilitar a leitura no console.

Veja um exemplo de `persistence.xml`:

xml

```
<persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence" version="2.2">
  <persistence-unit name="myPersistenceUnit" transaction-type="RESOURCE_LOCAL">
    <!-- Configuração do provedor JPA -->
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>

    <!-- Configurações de conexão com o banco de dados -->
    <properties>
      <property name="javax.persistence.jdbc.driver"
value="com.mysql.cj.jdbc.Driver" />
      <property name="javax.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/mydatabase" />
      <property name="javax.persistence.jdbc.user" value="root" />
      <property name="javax.persistence.jdbc.password" value="password" />

      <!-- Configurações adicionais do provedor (neste caso, Hibernate) -->
      <property name="hibernate.dialect"
value="org.hibernate.dialect.MySQL8Dialect" />
      <property name="hibernate.hbm2ddl.auto" value="update" />
      <property name="hibernate.show_sql" value="true" />
      <property name="hibernate.format_sql" value="true" />
    </properties>
  </persistence-unit>
</persistence>
```

Além das configurações básicas, existem configurações avançadas que ajudam a otimizar o desempenho e a adaptar o JPA para casos de uso específicos.

- **Caching:** A maioria dos provedores JPA oferece caching de segundo nível, que armazena entidades entre sessões e permite que os dados sejam carregados do cache em vez de uma consulta ao banco de dados.
- **Pooling de Conexões:** Para reduzir a sobrecarga de criação e encerramento de conexões, podemos configurar um pool de conexões.
- **Gerenciamento de Transações:** O JPA permite que o gerenciamento de transações seja configurado no persistence.xml por meio do atributo transaction-type da unidade de persistência.
 - **RESOURCE_LOCAL:** Usado para transações locais controladas pelo próprio EntityManager.
 - **JTA** (Java Transaction API): Usado para transações distribuídas, geralmente em servidores de aplicação, onde o JTA controla as transações entre múltiplas fontes de dados.
- **Configuração de Fetch:** O FetchType define como as entidades relacionadas serão carregadas, podendo ser LAZY (sob demanda) ou EAGER (antecipado). Esse comportamento é configurado diretamente nas entidades, mas é importante configurar a implementação para otimizar o carregamento de dados.

(FCC/MPE PB/2023) Em uma classe de acesso a dados de uma aplicação web Java que utiliza JPA, em condições ideais, a linha abaixo faz parte de um método que gerencia a conexão com um banco de dados.

```
EntityManagerFactory conn = Persistence.createEntityManagerFactory("MPEPB123PU");
```

A existência dessa linha na aplicação indica que no arquivo persistence.xml deve haver, dentre outras, a linha

- a) <factory-unit name="MPEPB123PU" transaction-type="JTA">
- b) <persistence-unit name="MPEPB123PU" transaction-type="RESOURCE_LOCAL">
- c) <database-unit name="MPEPB123PU" driver-type="com.mysql.cj.jdbc.Driver">
- d) <connection-unit database="MPEPB123PU" transaction-type="JTA">
- e) <property name="MPEPB123PU" transaction-type="RESOURCE_LOCAL">

Comentários:

Anotações

Várias questões cobram as anotações – então, vamos novamente trazê-las a você. Aqui teremos uma tabela mais completa e que ajudará a responder boa parte das questões que você encarar pela frente.

Anotação	Descrição
@Entity	Define uma classe como uma entidade JPA que será mapeada para uma tabela no banco de dados.
@Table	Especifica o nome da tabela do banco de dados para a entidade.
@Id	Define o campo como a chave primária da entidade.
@GeneratedValue	Configura a geração automática de valores para a chave primária.
@Column	Define o mapeamento de um campo para uma coluna da tabela.
@OneToOne	Configura um relacionamento de um-para-um entre entidades.
@OneToMany	Define um relacionamento de um-para-muitos entre entidades.
@ManyToOne	Configura um relacionamento de muitos-para-um entre entidades.
@ManyToMany	Define um relacionamento de muitos-para-muitos entre entidades.
@JoinColumn	Especifica a coluna de junção para relacionamentos.
@JoinTable	Configura uma tabela de junção para relacionamentos muitos-para-muitos.
@Embedded	Incorpora uma classe embutida (com @Embeddable) na entidade.
@Embeddable	Marca uma classe como embutível em uma entidade.
@EmbeddedId	Define uma chave primária composta embutida.
@IdClass	Define uma classe externa para uma chave primária composta.
@Transient	Indica que um campo não deve ser persistido no banco de dados.
@NamedQuery	Define uma consulta JPQL com nome, que pode ser reutilizada.
@NamedQueries	Define múltiplas consultas nomeadas na mesma entidade.
@NamedNativeQuery	Define uma consulta SQL nativa com nome, para uso com a entidade.
@NamedNativeQueries	Define múltiplas consultas SQL nativas nomeadas na mesma entidade.
@SequenceGenerator	Configura um gerador de sequência para a chave primária.
@TableGenerator	Define uma tabela para geração de chaves primárias.
@Version	Marca um campo de versão para controle de concorrência otimista.

@Inheritance	Configura a estratégia de herança de entidade (ex.: JOINED, SINGLE_TABLE).
@DiscriminatorColumn	Define uma coluna discriminadora para herança.
@DiscriminatorValue	Especifica o valor da coluna discriminadora para uma entidade em uma hierarquia de herança.
@Access	Define o tipo de acesso (FIELD ou PROPERTY) para persistência da entidade.
@Cacheable	Indica se a entidade deve ser armazenada em cache.
@OrderBy	Especifica a ordenação de uma coleção de entidades associadas.
@OrderColumn	Define uma coluna para manter a ordem de uma coleção indexada.
@MapsId	Associa um campo em uma entidade a uma chave primária em uma relação de um-para-um ou muitos-para-um.
@MapKey	Define o campo ou propriedade que serve como chave em um relacionamento Map com uma coleção.
@MapKeyColumn	Especifica a coluna usada para armazenar a chave em um Map.
@MapKeyJoinColumn	Define uma coluna de junção usada como chave em um relacionamento Map.
@Lob	Indica que um campo ou propriedade é do tipo LOB (Large Object), usado para armazenar dados grandes, como BLOB ou CLOB.
@Basic	Configura opções de carregamento e propriedades básicas de um atributo mapeado.
@Enumerated	Especifica como um tipo enumerado deve ser armazenado (ORDINAL ou STRING).
@Temporal	Define a precisão temporal de um campo Date ou Calendar (DATE, TIME ou TIMESTAMP).
@ElementCollection	Define uma coleção de elementos embutidos ou valores básicos associados a uma entidade.
@AttributeOverride	Permite modificar atributos de mapeamento em uma classe embutida.
@AttributeOverrides	Permite definir múltiplos @AttributeOverride para uma classe embutida.
@PostLoad	Método de callback executado após o carregamento da entidade.
@PrePersist	Método de callback executado antes de uma entidade ser persistida.
@PostPersist	Método de callback executado após uma entidade ser persistida.

@PreUpdate	Método de callback executado antes de uma entidade ser atualizada.
@PostUpdate	Método de callback executado após uma entidade ser atualizada.
@PreRemove	Método de callback executado antes de uma entidade ser removida.
@PostRemove	Método de callback executado após uma entidade ser removida.

QUESTÕES COMENTADAS

01. (FCC/MPE PB/2023) Em uma classe de acesso a dados de uma aplicação web Java que utiliza JPA, em condições ideais, a linha abaixo faz parte de um método que gerencia a conexão com um banco de dados.

```
EntityManagerFactory conn = Persistence.createEntityManagerFactory("MPEPB123PU");
```

A existência dessa linha na aplicação indica que no arquivo persistence.xml deve haver, dentre outras, a linha

- a) <factory-unit name="MPEPB123PU" transaction-type="JTA">
- b) <persistence-unit name="MPEPB123PU" transaction-type="RESOURCE_LOCAL">
- c) <database-unit name="MPEPB123PU" driver-type="com.mysql.cj.jdbc.Driver">
- d) <connection-unit database="MPEPB123PU" transaction-type="JTA">
- e) <property name="MPEPB123PU" transaction-type="RESOURCE_LOCAL">

Comentários:

A linha `EntityManagerFactory conn = Persistence.createEntityManagerFactory("MPEPB123PU");` indica que o `EntityManagerFactory` está sendo criado com uma unidade de persistência chamada "MPEPB123PU". No arquivo `persistence.xml`, as unidades de persistência são definidas pelo elemento `<persistence-unit>`, que configura o nome e o tipo de transação.

Gabarito: Letra B

02. (FCC/MPE PB/2023) Em uma aplicação Java que utiliza JPA, em condições ideais, o método da interface `EntityManager` utilizado para salvar um objeto de uma entidade da aplicação chamada Processo na tabela processo do banco de dados é o

- a) `persist`.
- b) `save`.
- c) `append`.
- d) `add`.
- e) `execute`.

Comentários:

Quando a banca afirma que queremos "salvar um objeto de uma entidade", ela está dizendo que queremos persistir esse objeto – o que é feito a partir do método `persist()`.

Gabarito: Letra A

03. (FCC/MPE PB/2023) A anotação `@Audited` em uma classe de entidade JPA indica o uso

- a) da biblioteca Flyway.
- b) do Hibernate Envers.

- c) do Spring Eureka.
- d) de Map Struct.
- e) do Swagger.

Comentários:

A anotação @Audited indica o uso do Hibernate Envers. Ela aponta que a entidade está marcada para ser auditada.

Gabarito: Letra B

04. (FGV/DPE RS/2023) Quando o J2EE foi substituído pelo JEE5, e as configurações via XML passaram a ser efetuadas com base em anotações, as entidades anotadas do JPA passaram a ser utilizadas no lugar dos Entity Beans.

Para definir uma chave primária com geração automática de valores, de forma independente do tipo de banco de dados, a configuração deve ser:

- (A) @Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
- (B) @Column(name="Id")
@Sequence(name="seq_chaves")
- (C) @Column(name="Id")
@Table(name="tab_chaves")
- (D) @Id
@GeneratedValue(strategy=GenerationType.SEQUENCE,
generator="sequenceGen")
@SequenceGenerator(name="sequenceGen",
sequenceName="seq_chaves")
- (E) @Id
@GeneratedValue(strategy = GenerationType.TABLE,
generator = "tableGen")
@TableGenerator(name = "tableGen", table = "tab_chaves",
pkColumnName = "id", pkColumnValue = "tabelaX",
valueColumnName = "valor")

Comentários:

A resposta correta para definir uma chave primária com geração automática de valores de forma independente do tipo de banco de dados é a letra E. Queremos uma anotação que irá apontar o uso de uma estratégia abordando uma tabela independente do banco de dados para a geração de chaves – que nos indica o uso de strategy=GenerationType.TABLE – dentro da anotação @GeneratedValue. A única alternativa que aborda essa anotação corretamente é a letra E. O generator = “tableGen” permite especificar o nome da tabela geradora, colunas e outros detalhes acerca dela.

Gabarito: Letra E

05. (FGV/DPE RS/2023) No contexto do Spring Data JPA, analise a declaração a seguir.

```
public interface UserRepository extends JpaRepository<User, Long>
{
    @Query("select codigo from User u where u.emailAddress = ?1")
    User findByEmailAddress(String emailAddress);
}
```

Supondo-se que todas as definições complementares à declaração foram devidamente estabelecidas, é correto afirmar que o trecho '?1' expressa:

- a) uma referência a uma variável de ambiente declarada e definida no Spring Data JPA;
- b) uma expressão regular a ser aplicada na execução do comando 'select';
- c) um código que estabelece o uso da cláusula LIKE na execução do comando 'select';
- d) um código que provoca o disparo de um prompt para coletar o valor a ser utilizado na execução do comando 'select';
- e) o valor de um parâmetro utilizado na invocação do método associado à consulta.

Comentários:

Essa questão explora o uso do ?1. Ele é um parâmetro associado à consulta – mais especificamente, um parâmetro posicional. Esse tipo de parâmetro permite que valores sejam passados para a consulta no momento em que o método é chamado, substituindo ?1 pelo valor do primeiro argumento do método `findByEmailAddress(String emailAddress)`.

Gabarito: Letra E

06. (VUNESP/CM SOB/2023) No contexto do Java EE 6, a especificação que descreve como deve ser o comportamento dos *frameworks* de persistência Java é conhecida como

- a) JSF.
- b) JPA.
- c) Hibernate.
- d) JTA.
- e) JMS.

Comentários:

A especificação de persistência do Java é o JPA – Java Persistence API.

Gabarito: Letra B

07. (FUNDATEC/PROCERGS/2023) O trecho de código abaixo, em Java Persistence API (JPA) versão 2.2, apresenta a declaração de uma chave primária composta de uma entidade de um banco de dados.

@_____

```
public class PessoaId implements Serializable {  
    String nome;  
    String sobrenome;  
}  
  
@_____  
public class Pessoa {  
    @_____ PessoaId id;  
    String endereco;  
}
```

Assinale a alternativa que preenche, correta e respectivamente, as lacunas do trecho de código acima.

- a) Embeddable – Entity – EmbeddedId
- b) Instance – Entity – Primary
- c) Association – Table – Primary
- d) Instance – Table – EmbeddedId
- e) Embeddable – Entity – Primary

Comentários:

Precisamos identificar as anotações para cada classe. No contexto do JPA, para definir uma chave primária composta, utilizamos uma classe que agrupa os campos que compõem a chave e a anotamos com `@Embeddable`. Isso indica que essa classe pode ser embutida como parte de uma outra entidade para compor uma chave primária – preenchendo nosso primeiro campo.

No segundo campo, temos a anotação `@Entity`, indicando que se trata de uma entidade JPA, ou seja, uma classe persistente, equivalente a uma tabela no banco de dados. Por fim, o terceiro campo recebe a anotação `@EmbeddedID`, que informa que esse campo é uma chave primária composta.

Gabarito: Letra A

08. (DIRENS/Oficial Aeronáutica/2023) O Java Persistence API (JPA) define os padrões de persistência, condição fundamental para desenvolvimento de aplicativos orientados a objetos. Há, ainda, uma relação aos referenciamentos e aos tratamentos de eventos dele com o JavaServer.

Considerando o exposto acima, informe verdadeiro (V) ou falso (F) para as assertivas abaixo e, em seguida, marque a opção que apresenta a sequência correta.

- () A forma de utilização de API de critérios de metamodelos faz referência aos atributos por meio de expressões.
- () Na utilização de metamodelos, é possível obter a descrição de uma unidade persistente.
- () O API de critérios se apresenta como uma opção à escrita em Java Persistence Query Language.
- () A API de critérios baseada em strings é realizada por meio do recebimento das informações das propriedades das entidades.

- a) (V); (V); (V); (F).

- b) (F); (F); (F); (V).
- c) (V); (F); (V); (V).
- d) (F); (V); (F); (F).

Comentários:

Vamos analisar cada item.

() *A forma de utilização de API de critérios de metamodelos faz referência aos atributos por meio de expressões.*

Verdadeiro. A API de critérios (Criteria API) permite que atributos sejam referenciados por meio de expressões tipadas, utilizando o metamodelo. Esse metamodelo é gerado automaticamente para cada entidade, permitindo que os atributos sejam acessados de maneira segura e orientada a objetos.

() *Na utilização de metamodelos, é possível obter a descrição de uma unidade persistente.*

Falso. Não conseguimos obter a descrição de uma entidade que foi persistida.

() *O API de critérios se apresenta como uma opção à escrita em Java Persistence Query Language.*

Verdadeiro. A Criteria API é uma alternativa à JPQL para criar consultas de forma programática. Ela é totalmente orientada a objetos e evita o uso de strings para construir consultas, oferecendo uma maneira tipada e segura de construir consultas dinâmicas em JPA.

() *A API de critérios baseada em strings é realizada por meio do recebimento das informações das propriedades das entidades.*

Verdadeiro. De fato, na Criteria API temos o recebimento das propriedades das entidades para definir os critérios.

Portanto, ficamos com V-F-V-V.

Gabarito: Letra C

09. (FCC/TRT 12/2023) Em uma aplicação Java que utiliza JPA, em condições ideais, considere manager um objeto válido criado por meio da instrução abaixo.

```
EntityManager manager =  
Persistence.createEntityManagerFactory("trtPU").createEntityManager();
```

A instrução `manager.createNamedQuery("Acesso.findByUserSenha", Acesso.class);` irá retornar um objeto do tipo

- a) `List<Acesso>`
- b) `Acesso`
- c) `Query<Acesso>`
- d) `TypedQuery<Acesso>`
- e) `ArrayList<Acesso>`

Comentários:

Sabemos que estamos fazendo uma consulta – mais especificamente, criando uma Named Query. Nos resta saber se a classe utilizada é Query ou TypedQuery. Para essa identificação, precisamos observar o uso do segundo parâmetro – `Acesso.class`. Como estamos definindo uma tipagem, podemos classificar o objeto de retorno como `TypedQuery<Acesso>`.

Gabarito: Letra D

10. (FURB/Pref. Blumenau/2022) O JavaTM Persistence API (JPA) fornece um mecanismo para gerenciar a persistência e mapeamento relacional de objeto e funções para as especificações de EJB. Registre V, para verdadeiro, ou F, para falso:

- () A JPA representa uma simplificação do modelo de programação de persistência.
- () A especificação JPA define explicitamente o mapeamento relacional de objetos, em vez de depender das implementações de mapeamento específicas do fornecedor.
- () A JPA padroniza a importante tarefa de mapeamento relacional de objetos, utilizando anotações ou o XML para mapear objetos para uma ou mais tabelas de um banco de dados.
- () A JPA foi projetada para operar dentro e fora de um contêiner Java Enterprise Edition (Java EE).
- () A JPA não fornece uma linguagem de consulta, portanto, não pode ser considerado uma linguagem de consulta EJB independente (também conhecida como JPQL).

Marque a alternativa que apresenta a sequência CORRETA:

- a) V, V, V, V, F.
- b) V, V, V, F, F.
- c) V, V, V, V, V.
- d) F, V, F, V, V.
- e) V, F, V, V, F.

Comentários:

Vamos analisar cada item.

() A JPA representa uma simplificação do modelo de programação de persistência.

Verdadeiro. A JPA foi criada para simplificar a persistência em Java, oferecendo uma maneira mais direta e menos complexa para desenvolver aplicações persistentes, em comparação com abordagens mais antigas, como os Entity Beans no EJB.

() A especificação JPA define explicitamente o mapeamento relacional de objetos, em vez de depender das implementações de mapeamento específicas do fornecedor.

Verdadeiro. A JPA estabelece um padrão para mapeamento objeto-relacional, definindo como as entidades devem ser mapeadas para tabelas no banco de dados, independentemente das implementações específicas dos fornecedores.

() A JPA padroniza a importante tarefa de mapeamento relacional de objetos, utilizando anotações ou o XML para mapear objetos para uma ou mais tabelas de um banco de dados.

Verdadeiro. A JPA permite o mapeamento de classes para tabelas usando anotações ou XML, padronizando essa tarefa e oferecendo flexibilidade para os desenvolvedores escolherem o método preferido.

(__)*A JPA foi projetada para operar dentro e fora de um contêiner Java Enterprise Edition (Java EE).*

Verdadeiro. Podemos usar a JPA tanto em um ambiente gerenciado (dentro de um contêiner Java EE) quanto em um ambiente não gerenciado (fora de um contêiner Java EE).

(__)*A JPA não fornece uma linguagem de consulta, portanto, não pode ser considerado uma linguagem de consulta EJB independente (também conhecida como JPQL).*

Falso. A JPA inclui a Java Persistence Query Language (JPQL), uma linguagem de consulta orientada a objetos que permite realizar operações de busca e manipulação de dados nas entidades mapeadas. Portanto, a afirmação de que a JPA não fornece uma linguagem de consulta está incorreta.

Ficamos com V-V-V-V-F.

Gabarito: Letra A

11. (FURB/Pref. Blumenau/2022) Sobre o JPA, analise as afirmativas:

I-JPA é um framework leve, baseado em POJOS (Plain Old Java Objects) para persistir objetos Java.

II-A Java Persistence API, diferente do que muitos imaginam, não é apenas um framework para Mapeamento Objeto-Relacional (ORM - Object-Relational Mapping), ela também oferece diversas funcionalidades essenciais em qualquer aplicação corporativa.

III-JPA provê limitadas funcionalidades para os programadores e também não representa uma simplificação do modelo de programação de persistência. A especificação JPA remove explicitamente o mapeamento relacional de objetos, em vez de depender das implementações de mapeamento específicas do fornecedor.

Estão CORRETAS as afirmativas:

- a) III, apenas.
- b) I, apenas.
- c) I e II, apenas.
- d) I, II e III.
- e) II, apenas.

Comentários:

Vamos analisar os itens.

I – Certo. O JPA é baseado em POJOS e seu objetivo é, de fato, persistir objetos Java.

II – Certo. O JPA oferece vários recursos que vão além de um simples ORM, com recursos como cache, gerenciamento de ciclo de vida, etc.

III – Errado. Fazer persistência de forma “tradicional”, com CRUD, é muito complexo – o JPA acaba sendo uma forma bem simplificada de se fazer isso. Além disso, não há remoção do mapeamento relacional.

Ficamos com I e II como corretas.

12. (FGV/Sefaz AM/2022) Java Persistence API (JPA) é um recurso da linguagem Java que descreve uma interface comum para *frameworks* de persistência de dados.

Analise o código a seguir da classe *User* especificada como uma entidade JPA.

```
1 @Entity(name="USER")
2 public class User {
3
4     @Id @GeneratedValue(strategy = GenerationType.AUTO,
5                          generator = "native")
6     @GenericGenerator(name = "native", strategy = "native")
7     @Column(name = "ID")
8     private Long id;
9
10    @Column(name="USER_NAME")
11    private String userName;
12
13    @Column(name="CREATED_TIME")
14    private Date creationTime;
15 }
16
```

O atributo *creationTime* é responsável por guardar a data e a hora de criação da entidade *User*.

A anotação do JPA, versão 2.0, que deve ser usada para anotar *creationTime* para especificar que este atributo deve ser armazenado no banco de dados com precisão timestamp, é

- a) @Basic.
- b) @Datetime.
- c) @Embeddable.
- d) @Lob.
- e) @Temporal.

Comentários:

Queremos armazenar a informação com precisão temporal. Vamos analisar cada alternativa.

- a) Errado. @Basic define atributos básicos, mas não especifica a precisão temporal.
- b) Errado. Não existe uma anotação @Datetime no JPA.
- c) Errado. @Embeddable indica que uma classe é embutível em outra, não está relacionada com datas.
- d) Errado. @Lob indica que o atributo é um Large Object (LOB), usado para armazenar dados binários ou texto grande.
- e) Certo. É a nossa resposta. @Temporal define a precisão temporal de um campo Date ou Calendar.

Portanto, correta a letra E.

13. (FGV/TJ TO/2022) A técnica em informática Ana está implementando o módulo de persistência da aplicação TJTOApp utilizando o JPA (Java Persistence API).

Observe abaixo o trecho principal da classe Departamento de TJTOApp:

```
@Entity
@Table(name="DEPART")
public class Departamento {

    @Id
    @Column(name="ID_DEPART")
    private Integer id;

    @Column(name="NM_DEPART")
    private String nome;

    private Integer qtdPessoas;
}
```

Ana precisa que a propriedade qtdPessoas da classe Departamento seja calculada pela aplicação e não seja persistida na tabela DEPART. Para especificar que a propriedade qtdPessoas seja ignorada pelo mecanismo de persistência JPA, Ana deve adicionar à qtdPessoas a anotação do JPA:

- a) @GeneratedValue;
- b) @Transient;
- c) @Temporal;
- d) @Enumerated;
- e) @Column(nullable = true).

Comentários:

No caso do campo qtdPessoas, Ana deseja que ele seja calculado pela aplicação e não seja armazenado no banco de dados. Portanto, a anotação @Transient é apropriada para esse propósito.

Ela é usada para indicar que um campo de uma entidade não deve ser persistido no banco de dados. Quando aplicamos @Transient a um campo, ele é ignorado pelo mecanismo de persistência e não será mapeado para uma coluna na tabela correspondente.

Gabarito: Letra B

14. (FCC/TRT 4/2022) As classes persistentes da JPA que são definidas por anotações são javax.persistence.Embeddable, javax.persistence.MappedSuperclass e

- a) javax.persistence.Interface
- b) javax.persistence.AbstractClass
- c) javax.persistence.MainClass
- d) javax.persistence.NamedClass
- e) javax.persistence.Entity

Comentários:

No JPA, as classes persistentes que podem ser mapeadas e configuradas por meio de anotações incluem:

- `javax.persistence.Entity`: Marca uma classe como uma entidade persistente, que é mapeada para uma tabela no banco de dados.
- `javax.persistence.Embeddable`: Define uma classe que pode ser incorporada (embutida) em outras entidades, mas que não possui uma tabela própria.
- `javax.persistence.MappedSuperclass`: Define uma classe que fornece mapeamentos de propriedades para subclasses, mas que não é uma entidade por si mesma.

Portanto, correta a letra E.

Gabarito: Letra E

15. (FCC/TRT 19/2022) Considere a classe abaixo em uma aplicação que utiliza JPA e Hibernate.

```
@Entity
public class Cliente {
    @Id
    @GeneratedValue(strategy = GenerationType. ...)
    private Long id;
    // atributos e métodos restantes
}
```

Para informar ao provedor de persistência que os valores a serem atribuídos ao identificador único (id) serão gerados pela coluna de autoincremento da tabela do banco de dados, utiliza-se, na lacuna I, o valor

- a) TABLE
- b) AUTO_GENERATED
- c) AUTO_INCREMENT
- d) IDENTITY
- e) SELF_GENERATED

Comentários:

Para gerarmos um valor auto incremental, usamos a anotação o valor INDETITY. Lembrando as opções que temos:

- **GenerationType.AUTO**: Delega ao provedor JPA a escolha da estratégia mais apropriada, com base nas configurações do banco de dados.
- **GenerationType.IDENTITY**: Usa a geração de IDs autoincrementados do banco de dados, comum em SGBDs como MySQL.
- **GenerationType.SEQUENCE**: Requer o uso de sequências do banco de dados para gerar IDs únicos, suportado por bancos como Oracle e PostgreSQL.
- **GenerationType.TABLE**: Usa uma tabela especial no banco de dados para simular a geração de uma sequência, garantindo independência em relação ao SGBD.

Portanto, correta a letra D.

16. (IDECAN/TJ PI/2022) Java Persistence API (JPA) é uma API que define uma interface que serve de padrão para frameworks de persistência de dados, como por exemplo o Hibernate e o EclipseLink. Por estar relacionada à linguagem Java, JPA possibilita o mapeamento objeto-relacional para Plain Old Java Objects (POJO). Em JPA utilizamos anotações para definir características que possibilitem o mapeamento objeto-relacional.

Marque a alternativa que indica, respectivamente, o nome da anotação JPA que determina que um dado atributo da entidade representa um tipo de objeto de grande volume e o nome da anotação JPA que determina que um dado atributo da entidade não deve ser persistido.

- a) @Blob e @ReadOnly
- b) @Large e @NotPersist
- c) @Blob e @NotPersist
- d) @BData e @ReadOnly
- e) @Lob e @Transient

Comentários:

Para indicarmos um objeto de grande volume – um Large Object – usamos a anotação @Lob, podendo especificar dados como dados binários (BLOB) ou texto longo (CLOB). Essa anotação é usada para mapear campos que contêm grandes volumes de dados. Já para marcar um atributo como não persistido, usamos a anotação @Transient, que faz com que o atributo seja ignorado pelo Manager.

17. (FCC/TRT 16/2022) Analise o código a seguir, que descreve a entidade Carro:

```
1 @Entity @Table(name="carros")
2 public class Carro{
3
4     @Id
5     private long id;
6
7     private String modelo;
8
9
10    private Collection<String> cores;
11 }
```

A anotação JPA, versão 2.0, a ser incluída na linha 9 para indicar que as cores da entidade Carro devem ser armazenadas em uma tabela separada da tabela carros, é

- a) @CollectionAttribute
- b) @ElementCollection
- c) @Embeddable
- d) @Embedded
- e) @InnerTable

Comentários:

A anotação para atender ao objetivo pretendido é a @ElementCollection. Essencialmente, ela permite que uma entidade tenha uma coleção de valores que são armazenados em uma tabela separada, relacionada à tabela principal da entidade. Essa tabela de coleção não possui uma entidade própria, mas é vinculada à entidade principal por uma chave estrangeira.

Gabarito: Letra B

18. (FGV/TRT 16/2022) Analise o código a seguir, que descreve a entidade Curso:

```
1 @Entity @Table(name="cursos")
2 public class Curso {
3
4     @Id
5     private long id;
6
7     private String nome;
8
9 }
```

A anotação JPA, versão 2.0, que deve ser incluída na linha 9 para indicar que o atributo totalMatriculados deve ser ignorado nas operações de persistência é

- a) @Column
- b) @Default
- c) @Ignore
- d) @OrphanRemoval
- e) @Transient

Comentários:

Como já vimos algumas vezes – para ignorar um atributo, de forma a não ser mais gerenciado pelo Manager, usamos a anotação @Transient.

Gabarito: Letra E

19. (FGV/TRT 13/2022) Analise o código a seguir, que descreve a entidade Departamento:

```
1 @Entity @Access(AccessType.FIELD)
2 @Table(name="departamentos")
3 public class Departamento {
4
5     @Id
6     private long id;
7
8     @Transient
9     private String nome;
10
11
12     public String getNome() {
13         return nome == null ? "" : nome.toUpperCase();
14     }
15
16 }
```

Assinale a opção que apresenta as anotações JPA 2.0 que devem ser incluídas na linha 11 para que a JPA utilize o método `getNome()` como mapeamento para a coluna `nome` da tabela `departamentos` nas operações de persistência.

- a) `@Column(name="nome") @Access(AccessType.PROPERTY)`
- b) `@Basic(field="nome") @Access(AccessType.PROPERTY)`
- c) `@Basic(field="nome") @Access(AccessType.FIELD)`
- d) `@Column(name="nome") @Access(AccessType.FIELD)`
- e) `@Column(field="nome") @Access(AccessType.FIELD)`

Comentários:

Para que a JPA utilize o método `getNome()` como mapeamento para a coluna `nome` da tabela `departamentos`, devemos configurar a acessibilidade baseada em propriedade. Por padrão, a JPA acessa os atributos diretamente (como está configurado com `@Access(AccessType.FIELD)` na classe), mas para utilizar o método `getNome()` como acesso, precisamos alterar a acessibilidade para `PROPERTY` especificamente para o campo `nome`. Então, teremos:

- `@Column(name="nome")`: Define que o método `getNome()` será mapeado para a coluna `nome` no banco de dados.
- `@Access(AccessType.PROPERTY)`: Especifica que a JPA deve usar o método `getNome()` (acesso por propriedade) para persistir e carregar o valor do atributo `nome`, em vez de acessar diretamente o campo.

Portanto, correta a letra A.

Gabarito: Letra A

20. (FCC/TRT 14/2022) O uso da anotação `@Audited` antes do nome da classe de entidade do banco de dados, em uma aplicação que utiliza JPA, sinaliza ao

- a) *Flyway* que a entidade já foi auditada.
- b) *Hibernate Envers* que a entidade será alvo de auditoria.
- c) *Map Struct* que a entidade já foi auditada.
- d) *Spring Cloud* que a entidade registra *logs* de transações com o banco de dados.

e) *Service Discovery* o caminho do arquivo de auditoria.

Comentários:

A anotação `@Audited` marca uma entidade para ser alvo de uma auditoria a partir do Hibernate Envers.

Gabarito: Letra B

21. (FCC/TRT 14/2022) Em uma aplicação que utiliza JPA 2.0, a interface a partir da qual se pode mapear um conjunto de classes para um banco de dados particular, permitindo a criação de consultas, a busca de objetos e a inserção de objetos no banco de dados, é a interface

- a) EntityManagerBuilder.
- b) EntityTransaction.
- c) TypedQuery.
- d) EntityManager.
- e) PreparedStatement.

Comentários:

Questão tranquila. O objeto que faz o gerenciamento das entidades, seu ciclo de vida, incorporação, atualização etc., é o EntityManager.

Gabarito: Letra D

LISTA DE QUESTÕES

01. (FCC/MPE PB/2023) Em uma classe de acesso a dados de uma aplicação web Java que utiliza JPA, em condições ideais, a linha abaixo faz parte de um método que gerencia a conexão com um banco de dados.

```
EntityManagerFactory conn = Persistence.createEntityManagerFactory("MPEPB123PU");
```

A existência dessa linha na aplicação indica que no arquivo persistence.xml deve haver, dentre outras, a linha

- a) <factory-unit name="MPEPB123PU" transaction-type="JTA">
- b) <persistence-unit name="MPEPB123PU" transaction-type="RESOURCE_LOCAL">
- c) <database-unit name="MPEPB123PU" driver-type="com.mysql.cj.jdbc.Driver">
- d) <connection-unit database="MPEPB123PU" transaction-type="JTA">
- e) <property name="MPEPB123PU" transaction-type="RESOURCE_LOCAL">

02. (FCC/MPE PB/2023) Em uma aplicação Java que utiliza JPA, em condições ideais, o método da interface EntityManager utilizado para salvar um objeto de uma entidade da aplicação chamada Processo na tabela processo do banco de dados é o

- a) persist.
- b) save.
- c) append.
- d) add.
- e) execute.

03. (FCC/MPE PB/2023) A anotação @Audited em uma classe de entidade JPA indica o uso

- a) da biblioteca Flyway.
- b) do Hibernate Envers.
- c) do Spring Eureka.
- d) de Map Struct.
- e) do Swagger.

04. (FGV/DPE RS/2023) Quando o J2EE foi substituído pelo JEE5, e as configurações via XML passaram a ser efetuadas com base em anotações, as entidades anotadas do JPA passaram a ser utilizadas no lugar dos Entity Beans.

Para definir uma chave primária com geração automática de valores, de forma independente do tipo de banco de dados, a configuração deve ser:

- (A) @Id
@GeneratedValue(strategy=GenerationType.IDENTITY)
- (B) @Column(name="Id")
@Sequence(name="seq_chaves")
- (C) @Column(name="Id")
@Table(name="tab_chaves")
- (D) @Id
@GeneratedValue(strategy=GenerationType.SEQUENCE,
generator="sequenceGen")
@SequenceGenerator(name="sequenceGen",
sequenceName=" seq_chaves")
- (E) @Id
@GeneratedValue(strategy = GenerationType.TABLE,
generator = "tableGen")
@TableGenerator(name = "tableGen", table = "tab_chaves",
pkColumnName = "id", pkColumnValue = "tabelaX",
valueColumnName = "valor")

05. (FGV/DPE RS/2023) No contexto do Spring Data JPA, analise a declaração a seguir.

```
public interface UserRepository extends JpaRepository<User, Long>
{
    @Query("select codigo from User u where u.emailAddress = ?1")
    User findByEmailAddress(String emailAddress);
}
```

Supondo-se que todas as definições complementares à declaração foram devidamente estabelecidas, é correto afirmar que o trecho '?1' expressa:

- a) uma referência a uma variável de ambiente declarada e definida no Spring Data JPA;
- b) uma expressão regular a ser aplicada na execução do comando 'select';
- c) um código que estabelece o uso da cláusula LIKE na execução do comando 'select';
- d) um código que provoca o disparo de um prompt para coletar o valor a ser utilizado na execução do comando 'select';
- e) o valor de um parâmetro utilizado na invocação do método associado à consulta.

06. (VUNESP/CM SOB/2023) No contexto do Java EE 6, a especificação que descreve como deve ser o comportamento dos *frameworks* de persistência Java é conhecida como

- a) JSF.
- b) JPA.

- c) Hibernate.
- d) JTA.
- e) JMS.

07. (FUNDATEC/PROCERGS/2023) O trecho de código abaixo, em Java Persistence API (JPA) versão 2.2, apresenta a declaração de uma chave primária composta de uma entidade de um banco de dados.

```
@_____
public class Pessoa implements Serializable {
    String nome;
    String sobrenome;
}

@_____
public class Pessoa {
    @_____ Pessoa id;
    String endereco;
}
```

Assinale a alternativa que preenche, correta e respectivamente, as lacunas do trecho de código acima.

- a) Embeddable – Entity – EmbeddedId
- b) Instance – Entity – Primary
- c) Association – Table – Primary
- d) Instance – Table – EmbeddedId
- e) Embeddable – Entity – Primary

08. (DIRENS/Oficial Aeronáutica/2023) O Java Persistence API (JPA) define os padrões de persistência, condição fundamental para desenvolvimento de aplicativos orientados a objetos. Há, ainda, uma relação aos referenciamentos e aos tratamentos de eventos dele com o JavaServer.

Considerando o exposto acima, informe verdadeiro (V) ou falso (F) para as assertivas abaixo e, em seguida, marque a opção que apresenta a sequência correta.

- () A forma de utilização de API de critérios de metamodelos faz referência aos atributos por meio de expressões.
- () Na utilização de metamodelos, é possível obter a descrição de uma unidade persistente.
- () O API de critérios se apresenta como uma opção à escrita em Java Persistence Query Language.

() A API de critérios baseada em strings é realizada por meio do recebimento das informações das propriedades das entidades.

- a) (V); (V); (V); (F).
- b) (F); (F); (F); (V).
- c) (V); (F); (V); (V).
- d) (F); (V); (F); (F).

09. (FCC/TRT 12/2023) Em uma aplicação Java que utiliza JPA, em condições ideais, considere manager um objeto válido criado por meio da instrução abaixo.

```
EntityManager manager =  
Persistence.createEntityManagerFactory("trtPU").createEntityManager();
```

A instrução `manager.createNamedQuery("Acesso.findByUserSenha", Acesso.class);` irá retornar um objeto do tipo

- a) `List<Acesso>`
- b) `Acesso`
- c) `Query<Acesso>`
- d) `TypedQuery<Acesso>`
- e) `ArrayList<Acesso>`

10. (FURB/Pref. Blumenau/2022) O JavaTM Persistence API (JPA) fornece um mecanismo para gerenciar a persistência e mapeamento relacional de objeto e funções para as especificações de EJB. Registre V, para verdadeiro, ou F, para falso:

- () A JPA representa uma simplificação do modelo de programação de persistência.
- () A especificação JPA define explicitamente o mapeamento relacional de objetos, em vez de depender das implementações de mapeamento específicas do fornecedor.
- () A JPA padroniza a importante tarefa de mapeamento relacional de objetos, utilizando anotações ou o XML para mapear objetos para uma ou mais tabelas de um banco de dados.
- () A JPA foi projetada para operar dentro e fora de um contêiner Java Enterprise Edition (Java EE).
- () A JPA não fornece uma linguagem de consulta, portanto, não pode ser considerado uma linguagem de consulta EJB independente (também conhecida como JPQL).

Marque a alternativa que apresenta a sequência CORRETA:

- a) V, V, V, V, F.
- b) V, V, V, F, F.
- c) V, V, V, V, V.

- d) F, V, F, V, V.
- e) V, F, V, V, F.

11. (FURB/Pref. Blumenau/2022) Sobre o JPA, analise as afirmativas:

I-JPA é um framework leve, baseado em POJOS (Plain Old Java Objects) para persistir objetos Java.

II-A Java Persistence API, diferente do que muitos imaginam, não é apenas um framework para Mapeamento Objeto-Relacional (ORM - Object-Relational Mapping), ela também oferece diversas funcionalidades essenciais em qualquer aplicação corporativa.

III-JPA provê limitadas funcionalidades para os programadores e também não representa uma simplificação do modelo de programação de persistência. A especificação JPA remove explicitamente o mapeamento relacional de objetos, em vez de depender das implementações de mapeamento específicas do fornecedor.

Estão CORRETAS as afirmativas:

- a) III, apenas.
- b) I, apenas.
- c) I e II, apenas.
- d) I, II e III.
- e) II, apenas.

12. (FGV/Sefaz AM/2022) Java Persistence API (JPA) é um recurso da linguagem Java que descreve uma interface comum para *frameworks* de persistência de dados.

Analise o código a seguir da classe *User* especificada como uma entidade JPA.

```
1 @Entity(name="USER")
2 public class User {
3
4     @Id @GeneratedValue(strategy = GenerationType.AUTO,
5                          generator = "native")
6     @GenericGenerator(name = "native", strategy = "native")
7     @Column(name = "ID")
8     private Long id;
9
10    @Column(name="USER_NAME")
11    private String userName;
12
13    @Column(name="CREATED_TIME")
14    private Date creationTime;
15 }
16
```

O atributo *creationTime* é responsável por guardar a data e a hora de criação da entidade *User*.

A anotação do JPA, versão 2.0, que deve ser usada para anotar *creationTime* para especificar que este atributo deve ser armazenado no banco de dados com precisão timestamp, é

- a) @Basic.
- b) @Datetime.
- c) @Embeddable.
- d) @Lob.
- e) @Temporal.

13. (FGV/TJ TO/2022) A técnica em informática Ana está implementando o módulo de persistência da aplicação TJTOApp utilizando o JPA (Java Persistence API).

Observe abaixo o trecho principal da classe Departamento de TJTOApp:

```
@Entity
@Table(name="DEPART")
public class Departamento {

    @Id
    @Column(name="ID_DEPART")
    private Integer id;

    @Column(name="NM_DEPART")
    private String nome;

    private Integer qtdPessoas;
}
```

Ana precisa que a propriedade qtdPessoas da classe Departamento seja calculada pela aplicação e não seja persistida na tabela DEPART. Para especificar que a propriedade qtdPessoas seja ignorada pelo mecanismo de persistência JPA, Ana deve adicionar à qtdPessoas a anotação do JPA:

- a) @GeneratedValue;
- b) @Transient;
- c) @Temporal;
- d) @Enumerated;
- e) @Column(nullable = true).

14. (FCC/TRT 4/2022) As classes persistentes da JPA que são definidas por anotações são javax.persistence.Embeddable, javax.persistence.MappedSuperclass e

- a) javax.persistence.Interface
- b) javax.persistence.AbstractClass
- c) javax.persistence.MainClass

- d) javax.persistence.NamedClass
- e) javax.persistence.Entity

15. (FCC/TRT 19/2022) Considere a classe abaixo em uma aplicação que utiliza JPA e Hibernate.

```
@Entity
public class Cliente {
    @Id
    @GeneratedValue(strategy = GenerationType. ...)
    private Long id;
    // atributos e métodos restantes }
```

Para informar ao provedor de persistência que os valores a serem atribuídos ao identificador único (id) serão gerados pela coluna de autoincremento da tabela do banco de dados, utiliza-se, na lacuna I, o valor

- a) TABLE
- b) AUTO_GENERATED
- c) AUTO_INCREMENT
- d) IDENTITY
- e) SELF_GENERATED

16. (IDECAN/TJ PI/2022) Java Persistence API (JPA) é uma API que define uma interface que serve de padrão para frameworks de persistência de dados, como por exemplo o Hibernate e o EclipseLink. Por estar relacionada à linguagem Java, JPA possibilita o mapeamento objeto-relacional para Plain Old Java Objects (POJO). Em JPA utilizamos anotações para definir características que possibilitem o mapeamento objeto-relacional.

Marque a alternativa que indica, respectivamente, o nome da anotação JPA que determina que um dado atributo da entidade representa um tipo de objeto de grande volume e o nome da anotação JPA que determina que um dado atributo da entidade não deve ser persistido.

- a) @Blob e @ReadOnly
- b) @Large e @NotPersist
- c) @Blob e @NotPersist
- d) @BData e @ReadOnly
- e) @Lob e @Transient

17. (FCC/TRT 16/2022) Analise o código a seguir, que descreve a entidade Carro:

```
1 @Entity @Table(name="carros")
2 public class Carro{
3
4     @Id
5     private long id;
6
7     private String modelo;
8
9
10    private Collection<String> cores;
11 }
```

A anotação JPA, versão 2.0, a ser incluída na linha 9 para indicar que as cores da entidade Carro devem ser armazenadas em uma tabela separada da tabela carros, é

- a) @CollectionAttribute
- b) @ElementCollection
- c) @Embeddable
- d) @Embedded
- e) @InnerTable

18. (FGV/TRT 16/2022) Analise o código a seguir, que descreve a entidade Curso:

```
1 @Entity @Table(name="cursos")
2 public class Curso {
3
4     @Id
5     private long id;
6
7     private String nome;
8
9     private int totalMatriculados;
```

A anotação JPA, versão 2.0, que deve ser incluída na linha 9 para indicar que o atributo totalMatriculados deve ser ignorado nas operações de persistência é

- a) @Column
- b) @Default
- c) @Ignore
- d) @OrphanRemoval
- e) @Transient

19. (FGV/TRT 13/2022) Analise o código a seguir, que descreve a entidade Departamento:

```
1 @Entity @Access(AccessType.FIELD)
2 @Table(name="departamentos")
3 public class Departamento {
4
5     @Id
6     private long id;
7
8     @Transient
9     private String nome;
10
11
12     public String getNome() {
13         return nome == null ? "" : nome.toUpperCase();
14     }
15
16 }
```

Assinale a opção que apresenta as anotações JPA 2.0 que devem ser incluídas na linha 11 para que a JPA utilize o método `getNome()` como mapeamento para a coluna `nome` da tabela `departamentos` nas operações de persistência.

- a) `@Column(name="nome") @Access(AccessType.PROPERTY)`
- b) `@Basic(field="nome") @Access(AccessType.PROPERTY)`
- c) `@Basic(field="nome") @Access(AccessType.FIELD)`
- d) `@Column(name="nome") @Access(AccessType.FIELD)`
- e) `@Column(field="nome") @Access(AccessType.FIELD)`

20. (FCC/TRT 14/2022) O uso da anotação `@Audited` antes do nome da classe de entidade do banco de dados, em uma aplicação que utiliza JPA, sinaliza ao

- a) *Flyway* que a entidade já foi auditada.
- b) *Hibernate Envers* que a entidade será alvo de auditoria.
- c) *Map Struct* que a entidade já foi auditada.
- d) *Spring Cloud* que a entidade registra *logs* de transações com o banco de dados.
- e) *Service Discovery* o caminho do arquivo de auditoria.

21. (FCC/TRT 14/2022) Em uma aplicação que utiliza JPA 2.0, a interface a partir da qual se pode mapear um conjunto de classes para um banco de dados particular, permitindo a criação de consultas, a busca de objetos e a inserção de objetos no banco de dados, é a interface

- a) `EntityManagerBuilder`.
- b) `EntityTransaction`.
- c) `TypedQuery`.
- d) `EntityManager`.

e) PreparedStatement.

■

GABARITO

GABARITO



1. Letra B
2. Letra A
3. Letra B
4. Letra E
5. Letra E
6. Letra B
7. Letra A

8. Letra C
9. Letra D
10. Letra A
11. Letra C
12. Letra E
13. Letra B
14. Letra E

15. Letra D
16. Letra E
17. Letra B
18. Letra E
19. Letra A
20. Letra B
21. Letra D

HIBERNATE

Conceitos Gerais



Hibernate é um **framework de código aberto para ORM** (Mapeamento Objeto-Relacional), destinado **primariamente ao ecossistema Java**, porém presente também em outros ambientes, como no .Net com o nome NHibernate. Criado pela Red Hat em 2001, seu objetivo é facilitar o mapeamento dos atributos entre uma base tradicional de dados relacionais e um código orientado a objetos, mediante o uso de arquivos XML ou anotações de Java.

No desenvolvimento de uma aplicação tradicional que interage com um banco de dados, seria necessário escrever manualmente comandos SQL para operações de criação, leitura, atualização e exclusão (CRUD), o que aumenta a quantidade de código, dificulta a manutenção e demanda um controle específico sobre cada banco de dados utilizado.

O conceito central do Hibernate é o Mapeamento Objeto-Relacional (ORM), que possibilita a representação de entidades do domínio de uma aplicação como classes Java. Assim, uma classe Produto em Java, com atributos como id, nome, e preço, pode ser diretamente mapeada para uma tabela produto no banco de dados, onde cada instância de Produto corresponde a uma linha nessa tabela. Para realizar esse mapeamento, o Hibernate utiliza anotações como `@Entity`, `@Id`, `@Column`, entre outras, que indicam ao framework como a classe Java e seus atributos devem se correlacionar com a estrutura da tabela.

Para implementar o ORM, usamos uma especificação chamada de **Java Persistence API (JPA)**. Ela fornece um conjunto de padrões para ORM no Java – mas, como é somente uma especificação, precisa de uma implementação para ser estabelecida, e o Hibernate é uma das implementações mais populares dessa especificação. Embora o Hibernate implemente os requisitos do JPA, ele também vai além ao oferecer funcionalidades exclusivas, como **cache de segundo nível**, que podem melhorar a performance ao minimizar o acesso repetido ao banco de dados.

(IBFC/TRF 5/2024) O sistema _____ é muito usado para o mapeamento de objetos Java para relações, ele oferece uma implementação do Java Persistence API (JPA). Assinale a alternativa que preencha corretamente a lacuna, tendo como tema a arquitetura de soluções em banco de dados para Web.

- a) Django
- b) Hibernate
- c) Alchemy
- d) Json

Comentários:

Para ORM e persistência usando a JPA, usamos o Hibernate. *(Gabarito: Letra B)*

Arquitetura

A arquitetura do Hibernate é projetada para permitir que desenvolvedores interajam de forma eficiente e organizada com bancos de dados relacionais, escondendo a complexidade das operações SQL por trás de uma camada de abstração baseada em objetos Java. Essa **arquitetura é composta por uma série de componentes** bem definidos, que gerenciam o ciclo de vida das entidades, as transações e o cache, possibilitando operações de leitura e manipulação de dados de forma otimizada.

No núcleo dessa arquitetura, encontram-se alguns elementos centrais: **SessionFactory**, **Session**, **Transaction**, **Query**, e **o sistema de cache**. Cada um desses componentes desempenha um papel crucial, e sua interação permite que o Hibernate gerencie eficientemente a persistência dos dados e o mapeamento objeto-relacional.

Componentes Hibernate

SessionFactory

Session

Transaction

Query

Cache

Vamos explorar cada um desses componentes.

SessionFactory

O **SessionFactory** é um dos elementos centrais do Hibernate e desempenha um papel fundamental na **configuração e gestão das Sessions**, que são as **instâncias responsáveis por executar operações de persistência e consulta** no banco de dados. Em uma aplicação Hibernate, o **SessionFactory** funciona como uma fábrica de **Sessions**. Ele **encapsula toda a configuração do Hibernate e também gerencia as informações sobre as entidades e o banco de dados**.

Uma das principais características do **SessionFactory** é seu **custo de criação relativamente alto**, pois ele carrega todas as configurações e metadados necessários para a persistência e para o mapeamento das classes Java para as tabelas do banco de dados. Por isso, o **SessionFactory é projetado para ser um singleton**, ou seja, uma única instância dele deve existir para toda a aplicação. Essa instância única é criada no momento da inicialização do aplicativo e reutilizada durante todo o seu ciclo de vida.

O **SessionFactory** é configurado com base em um arquivo de configuração XML, geralmente chamado `hibernate.cfg.xml`, ou por meio de anotações nas classes de entidade. Esse arquivo especifica diversas propriedades, como a URL de conexão do banco de dados, o nome de usuário

e a senha, além de parâmetros específicos do Hibernate, como dialeto SQL e configurações de cache. No código Java, podemos criar o `SessionFactory` usando a classe `Configuration`, que carrega o arquivo `hibernate.cfg.xml` e prepara o Hibernate para operar com o banco de dados especificado.

Java

```
Configuration configuration = new Configuration();  
configuration.configure("hibernate.cfg.xml");  
SessionFactory sessionFactory = configuration.buildSessionFactory();
```

Neste exemplo, o método `configure()` carrega o arquivo `hibernate.cfg.xml` e lê todas as informações necessárias para a configuração, como os detalhes do banco de dados e o mapeamento das entidades. O método `buildSessionFactory()` então cria a instância do `SessionFactory`.

O arquivo `hibernate.cfg.xml` é onde declaramos as configurações que o `SessionFactory` precisa para estabelecer a conexão com o banco de dados e reconhecer as entidades da aplicação. Uma vez configurado o `SessionFactory`, podemos usá-lo para criar sessões e interagir com o banco de dados.

Session

Session é o principal ponto de interação entre a aplicação e o banco de dados. Em outras palavras, ela **representa uma unidade de trabalho que permite executar operações de persistência e recuperação de dados**. Ao contrário do `SessionFactory`, que é projetado para ser uma instância única (*singleton*), a `Session` é uma instância leve, criada toda vez que precisamos interagir com o banco de dados. Essa separação permite que cada transação com o banco de dados tenha seu próprio ciclo de vida e escopo de atuação, tornando o gerenciamento de persistência mais eficiente e seguro.

A `Session` encapsula uma conexão com o banco de dados e gerencia as operações realizadas dentro desse contexto. Ela atua como uma **interface direta para operações de CRUD** (Create, Read, Update, Delete), permitindo que se manipulem objetos persistentes (ou seja, objetos Java que representam registros no banco de dados) sem a necessidade de escrever comandos SQL diretamente. As operações realizadas dentro de uma `Session` são transacionais, e seu escopo é definido pela transação atual. Isso significa que uma `Session` geralmente deve ser aberta, utilizada para realizar operações e, então, fechada ao término do trabalho para liberar recursos.

Você pode imaginar a `Session` como um “gerenciador de objetos” que coordena as operações em cada entidade e mantém o estado das mesmas durante o tempo em que está ativa. Esse gerenciamento de estado é uma das características fundamentais da `Session` e está diretamente relacionado ao ciclo de vida das entidades.

Aqui é importante que você se lembre do conceito de **fases de vida**, ou **estados de persistência**, das entidades – oriundas lá do JPA. Temos três estados no ciclo de vida de uma entidade:



- **Transient (Transiente):** uma entidade está no estado transiente quando foi criada, mas ainda não está associada a nenhuma Session e não foi persistida no banco de dados. Um objeto recém-inicializado está nesse estado.
- **Persistent (Persistente):** uma entidade entra no estado persistente quando é associada a uma Session e, portanto, o Hibernate a considera uma entidade gerenciada. Alterações feitas em uma entidade persistente são automaticamente sincronizadas com o banco de dados ao final de uma transação. Este é o estado ativo enquanto a Session está aberta.
- **Detached (Desanexado):** uma entidade torna-se desanexada quando sua Session é fechada ou quando é explicitamente removida do contexto persistente. Neste estado, a entidade não é mais gerenciada pelo Hibernate e alterações feitas nela não serão sincronizadas automaticamente com o banco de dados.

Estados das Entidades

Transient

Persistent

Detached

(FGV/PGM Niterói/2023) No contexto do Hibernate, considere a declaração a seguir.

```
Estudante estudante= new Estudante();
```

É correto afirmar que o objeto estudante, ao longo da sua existência/utilização, pode estar em algum dos estados:

- Defined, Instanced e Latent;
- Lazy, Eager e Restfull;
- On, Loaded e Off;
- Open, Active e Closed;
- Transient, Persistent e Detached.

Comentários:

Os três estados do objeto estudante são Transient, Persistent e Detached. (Gabarito: Letra E)

Como cada `Session` é leve, é comum que ela seja “esquecida” aberta e possa provocar vazamento de dados. Um padrão comum em aplicações Java é o uso de uma nova `Session` para cada requisição HTTP em aplicações web, garantindo que cada requisição seja tratada de forma independente. Isso é feito frequentemente em combinação com frameworks como o Spring, que fornece métodos de integração para gerenciar o ciclo de vida da `Session` de forma automática.

Bom, a `Session` permite que façamos operações CRUD e consultas no banco de dados. As operações mais comuns incluem:

- **Salvar uma entidade:** o método `save` insere uma nova entidade no banco de dados. O objeto é gerenciado pela `Session` e entra no estado persistente. A operação é concluída apenas quando a transação é confirmada.
- **Atualizar uma entidade:** o método `update` reanexa uma entidade desanexada ao contexto persistente da `Session`. Ao fazer isso, qualquer alteração na entidade será sincronizada com o banco de dados.
- **Excluir uma entidade:** o método `delete` remove uma entidade do banco de dados. Após a exclusão, o objeto entra no estado transiente, pois deixa de existir no banco de dados.
- **Buscar uma entidade:** o método `get` busca uma entidade com base em sua chave primária, e o método `load` também realiza essa busca, mas com comportamento ligeiramente diferente. O método `get` retorna null caso o objeto não seja encontrado, enquanto o `load` lança uma exceção se o objeto não estiver disponível.

Veja um exemplo:

Java

```
Session session = sessionFactory.openSession();
Transaction transaction = session.beginTransaction();

// Salvar uma nova entidade
Produto novoProduto = new Produto();
novoProduto.setNome("Lápis");
novoProduto.setPreco(0.75);
session.save(novoProduto); // produto é persistente após save()

// Buscar uma entidade
Produto produtoExistente = session.get(Produto.class, 1L);
if (produtoExistente != null) {
    System.out.println("Produto encontrado: " +
        produtoExistente.getNome());
}

// Excluir uma entidade
session.delete(produtoExistente);

transaction.commit();
session.close();
```

(FGV/Câmara dos Deputados/2023) No âmbito da arquitetura do Hibernate, assinale o objeto que estabelece uma conexão física com um banco de dados e é projetado para ser instanciado a cada vez que seja necessária uma interação com o mesmo.

- a) Criteria object.
- b) Query object.
- c) Session object.
- d) SessionFactory object.
- e) Transaction object.

Comentários:

Quando começamos a monitorar uma entidade no Hibernate, estabelecendo uma conexão com o banco de dados e "sincronizando" as informações, usamos um objeto do tipo Session. (Gabarito: Letra C)

Transaction

O **Transaction** é um componente usado para garantir que as **operações de manipulação de dados sejam realizadas de forma atômica e consistente** no banco de dados. Ele encapsula o

conceito de transação, que é uma unidade de trabalho que deve ser realizada de maneira completa ou revertida completamente em caso de falha. Isso garante a integridade dos dados, mesmo em cenários de falha ou erro, alinhando-se às propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade) dos sistemas de banco de dados relacionais.

Uma **transação representa um agrupamento lógico de uma ou mais operações** que **precisam ser tratadas como uma única unidade de trabalho**. Por exemplo, considere um sistema de banco onde uma transação precisa transferir dinheiro de uma conta para outra. Esse processo envolve duas operações: debitar um valor de uma conta e creditar o mesmo valor em outra. Ambas as operações precisam ser concluídas para que a transação seja válida. Se qualquer uma das operações falhar, a transação inteira deve ser revertida para evitar inconsistências, como dinheiro sendo debitado de uma conta sem ser creditado na outra.

No Hibernate, o componente `Transaction` facilita essa abordagem ao encapsular os detalhes de gerenciamento de transações do banco de dados e fornecer métodos para iniciar, confirmar e reverter transações. O uso explícito de `Transaction` é altamente recomendado, pois operações sem controle transacional podem levar a comportamentos imprevisíveis ou inconsistências nos dados.

Uma transação passa por 3 fases: inicialização, execução e confirmação – e, eventualmente, se houver erro, temos uma reversão. Cada uma dessas fases do ciclo de vida da transação são comandadas por um método:

- **Início da Transação:** A transação é iniciada explicitamente com o método `beginTransaction()` da `Session`. Isso sinaliza ao Hibernate que qualquer operação subsequente deve ser tratada como parte de uma unidade atômica.
- **Execução das Operações:** Todas as operações de manipulação de dados, como `save`, `update`, `delete` ou consultas, são executadas dentro do contexto da transação.
- **Confirmação da Transação:** Após o término bem-sucedido de todas as operações, a transação é confirmada com o método `commit()`. Isso instrui o Hibernate a aplicar permanentemente as mudanças no banco de dados.
- **Reversão da Transação:** Em caso de erro ou falha durante as operações, a transação pode ser revertida usando o método `rollback()`. Isso desfaz qualquer alteração realizada no contexto da transação, garantindo que o banco de dados retorne ao estado original.

Vamos ver como seria um código típico para implementação da transação.

Java

```
Session session = sessionFactory.openSession();
Transaction transaction = null;

try {
    // Início da transação
    transaction = session.beginTransaction();

    // Operações de manipulação de dados
    Produto produto1 = new Produto();
    produto1.setNome("Caneta");
    produto1.setPreco(1.50);

    Produto produto2 = new Produto();
    produto2.setNome("Lápis");
    produto2.setPreco(0.75);

    session.save(produto1);
    session.save(produto2);

    // Confirmação da transação
    transaction.commit();
} catch (Exception e) {
    if (transaction != null) {
        // Reversão da transação em caso de erro
        transaction.rollback();
    }
    e.printStackTrace();
} finally {
    // Fechamento da sessão
    session.close();
}
```

Query

Query é a interface responsável por **executar consultas no banco de dados** e retornar os resultados correspondentes. Ele permite que desenvolvedores recuperem, manipulem e filtrem dados utilizando linguagens de consulta como **HQL (Hibernate Query Language)**, **JPQL (Java Persistence Query Language)**, e até mesmo SQL nativo.

O componente Query é altamente flexível e uma parte essencial para implementar operações de leitura no Hibernate, oferecendo suporte tanto a consultas estáticas quanto dinâmicas, além de

possibilitar otimizações avançadas, como o uso de cache para resultados. Usamos o componente para definir, parametrizar e executar consultas que retornam entidades, coleções ou projeções específicas do banco de dados. Esse componente encapsula a lógica de consulta e abstrai os detalhes de implementação do banco, proporcionando uma interface simples para lidar com operações de recuperação.

Cache

O Cache é uma funcionalidade voltada para otimizar o desempenho de aplicações, reduzindo a quantidade de consultas ao banco de dados, e o tempo necessário para acessar dados usados frequentemente. Ele é implementado em dois níveis distintos: **cache de primeiro nível** e **cache de segundo nível**, ambos desempenhando papéis específicos no gerenciamento de entidades e resultados de consultas.

O **cache de primeiro nível** é o mais básico e **está sempre ativado no Hibernate**, sem necessidade de configuração explícita. Ele é **gerenciado diretamente pela instância da Session** e é exclusivo para cada sessão aberta. Isso significa que, enquanto a mesma **Session** estiver ativa, todas as entidades carregadas ou salvas durante seu ciclo de vida ficam armazenadas na memória dessa **Session**.

Quando a aplicação solicita uma entidade ao Hibernate, a **Session** verifica primeiro se a entidade já está presente no cache de primeiro nível. Caso esteja, ela é retornada diretamente da memória, evitando uma nova consulta ao banco de dados. Se não estiver, a **Session** realiza a consulta e, ao carregar a entidade, a adiciona ao cache de primeiro nível.

Já o **cache de segundo nível** é um cache opcional, e deve ser configurado explicitamente. Ele é **compartilhado entre múltiplas Sessions**, o que o torna **mais abrangente do que o cache de primeiro nível**. Enquanto o cache de primeiro nível opera no nível da **Session**, o cache de segundo nível é gerenciado pelo **SessionFactory** e pode armazenar entidades, coleções e resultados de consultas em memória ou em outras soluções de armazenamento.

O cache de segundo nível no Hibernate pode ser ativado e configurado por meio de provedores de cache, como:

- Ehcache
- Infinispan
- Redis
- Memcached

O cache de segundo nível funciona como uma **camada intermediária entre a aplicação e o banco de dados**. Quando uma entidade é carregada, o Hibernate verifica primeiro no cache de segundo nível (caso ele esteja ativado). Se a entidade estiver no cache, ela é retornada diretamente; caso contrário, uma consulta é feita ao banco e os dados são armazenados no cache para acessos

futuros. A principal diferença em relação ao cache de primeiro nível é que o cache de segundo nível é compartilhado entre múltiplas sessões e pode persistir além do ciclo de vida de uma Session.

O Hibernate oferece várias estratégias de cache para controlar como os dados são armazenados e acessados:

- **Read-Only:** Ideal para dados que nunca mudam, como tabelas de referência (ex.: lista de estados ou países).
- **Read-Write:** Permite leitura e escrita com controle de versões. Adequado para dados que podem mudar com frequência.
- **NonStrictReadWrite:** Permite leitura e escrita, mas sem controle de versões. Útil para cenários onde a consistência eventual é aceitável.
- **Transactional:** Suporta transações distribuídas, garantindo que os dados no cache estejam sempre consistentes.

Além do cache de entidades, o Hibernate também oferece um cache para os resultados de consultas, chamado **Query Cache**. Esse cache armazena o resultado de consultas específicas, incluindo os critérios de busca e a lista de entidades retornadas. Ele é especialmente útil em consultas frequentes e previsíveis, como buscas por listas de produtos ou categorias.

Consultas

O Hibernate, através do componente Query, suporta consulta em HQL (Hibernate Query Language), JPQL e SQL nativo. Vamos explorar brevemente a interação e as linguagens aqui.

HQL

O HQL é uma **linguagem específica do Hibernate**, projetada para trabalhar diretamente com entidades Java e seus atributos, em vez de tabelas e colunas do banco de dados. Ele abstrai os detalhes do SQL tradicional e permite que desenvolvedores escrevam consultas mais intuitivas e portáteis entre diferentes bancos de dados.

O HQL é baseado em SQL, mas opera no nível de objetos, o que significa que ele utiliza os nomes das classes e atributos do modelo de dados em vez dos nomes das tabelas e colunas no banco. Por exemplo, ao consultar uma entidade chamada **Produto** com atributos como **id**, **nome** e **preco**, você escreveria:

```
FROM Produto WHERE preco > 10.0
```



```
SELECT * FROM produtos WHERE preco > 10.0
```

JPQL

O JPQL é a linguagem de consulta oriunda da Java Persistence API, que permite a interação com bancos de dados usando uma abordagem orientada a objetos. Assim como o HQL no Hibernate, a JPQL opera sobre entidades e seus atributos, abstraindo o acesso às tabelas e colunas do banco de dados. A diferença principal é que a JPQL é padronizada, o que garante portabilidade entre implementações JPA, como Hibernate, EclipseLink, e OpenJPA.

Criteria API

A **Criteria API** é uma funcionalidade do JPA que **permite criar consultas de forma programática e dinâmica**. Ao contrário do HQL ou JPQL, onde as consultas são escritas como strings, a **Criteria API utiliza um modelo baseado em objetos**, o que facilita a construção de consultas em tempo de execução, especialmente quando os critérios de busca são definidos de forma dinâmica. Essa abordagem reduz o risco de erros de sintaxe, melhora a legibilidade e elimina a necessidade de concatenar strings para construir consultas complexas.

A Criteria API é baseada em três componentes principais:

- **CriteriaBuilder:** Um objeto que fornece métodos para criar critérios de consulta, filtros e expressões. Ele é obtido a partir do EntityManager.
- **CriteriaQuery:** Representa a consulta como um todo. É criado com base no CriteriaBuilder e define a estrutura da consulta (seleções, condições, agrupamentos, etc.).
- **Root:** Representa a entidade raiz da consulta e fornece acesso aos seus atributos.

Veja um exemplo de consulta com filtro:

Java

```
CriteriaBuilder cb = entityManager.getCriteriaBuilder();
CriteriaQuery<Produto> cq = cb.createQuery(Produto.class);
Root<Produto> root = cq.from(Produto.class);

// Adicionando a condição WHERE
cq.select(root).where(cb.greaterThan(root.get("preco"), 10.0));

TypedQuery<Produto> query = entityManager.createQuery(cq);
List<Produto> produtos = query.getResultList();
```

SQL

Embora o Hibernate promova a abstração do banco de dados, o uso de SQL nativo pode ser necessário nos seguintes casos:

- Consultas altamente otimizadas para bancos específicos, onde o SQL gerado pelo Hibernate não é eficiente.
- Acesso direto a tabelas que não estão mapeadas no modelo de entidades.
- Operações complexas que não são facilmente expressas em HQL ou JPQL.
- Integração com procedimentos armazenados ou funções específicas do banco.

Para implementarmos o SQL nativo, podemos usar o método `createNativeQuery` para criar as consultas, ou usando Named Native Queries, através de anotações e arquivos XML.

Gestão de Relacionamento

A gestão de relacionamentos é um dos aspectos centrais do framework, permitindo que entidades Java sejam associadas umas às outras de maneira natural, simulando as relações entre tabelas em um banco de dados relacional. Por meio de anotações ou configurações em XML, o Hibernate gerencia automaticamente os relacionamentos entre entidades, oferecendo suporte a diversos tipos de associações.

As principais associações seguem o padrão JPA:

- One-to-one (um para um)
- One-to-many (um para muitos)
- Many-to-one (muitos para um)
- Many-to-many (muitos para muitos)

Java

```
@Entity
public class Cliente {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "endereco_id") // Nome da coluna na tabela Cliente
    private Endereco endereco;
}
```

Cascade

O comportamento de **cascade** no Hibernate define como as operações realizadas em uma entidade principal devem ser propagadas para as entidades relacionadas. Isso evita que seja necessário gerenciar explicitamente as operações para cada entidade envolvida no relacionamento.

O Hibernate suporta diferentes tipos de propagação, configurados através da enumeração `CascadeType`:

- **`CascadeType.PERSIST`**: Propaga a operação de persistência (inserção) para as entidades relacionadas. Se a entidade principal for salva (save ou persist), as entidades associadas também serão salvas.

- **CascadeType.MERGE**: Propaga a operação de atualização para as entidades relacionadas. Se a entidade principal for mesclada (merge), as entidades associadas também serão mescladas.
- **CascadeType.REMOVE**: Propaga a operação de exclusão para as entidades relacionadas. Se a entidade principal for excluída (remove), as entidades associadas também serão excluídas.
- **CascadeType.REFRESH**: Propaga a operação de atualização para sincronizar o estado da entidade associada com o banco de dados.
- **CascadeType.DETACH**: Propaga a operação de "desanexar" (detach) para as entidades relacionadas. Isso remove as entidades do contexto de persistência.
- **CascadeType.ALL**: Combina todos os tipos acima (PERSIST, MERGE, REMOVE, REFRESH, DETACH).

Fetch

O comportamento de **fetch** no Hibernate define como os dados relacionados são carregados do banco de dados quando a entidade principal é acessada. O Hibernate oferece duas estratégias principais de carregamento:

Mapeamento Avançado

Mapeamento de Herança e Classes

O **mapeamento de herança e classes** no Hibernate é uma funcionalidade que permite modelar hierarquias de herança em entidades Java e persistir esses dados em tabelas relacionais. O Hibernate oferece várias estratégias para mapear classes que possuem herança, considerando os requisitos de desempenho, escalabilidade e flexibilidade do banco de dados. Esses mapeamentos são definidos por meio de anotações ou arquivos de configuração e determinam como as classes e seus atributos serão representados no modelo relacional.

Em um modelo orientado a objetos, a herança permite que uma classe (subclasse) herde atributos e métodos de outra classe (superclasse). No entanto, bancos de dados relacionais não possuem suporte nativo para herança. Para implementar essa funcionalidade, o Hibernate oferece três estratégias principais de mapeamento:

- **Table per Class Hierarchy** (Uma Tabela por Hierarquia de Classe): toda a hierarquia de classes é mapeada para uma única tabela no banco de dados. Essa tabela contém colunas para todos os atributos das classes da hierarquia e uma coluna discriminadora para identificar a classe específica de cada registro.
 - **Anotação:** `@Inheritance(strategy = InheritanceType.SINGLE_TABLE)`
- **Table per Subclass** (Uma Tabela por Subclasse): cada classe da hierarquia possui sua própria tabela. A tabela da superclasse contém os atributos comuns a todas as subclasses, enquanto as tabelas das subclasses contém os atributos específicos de cada uma. Joins são usados para recuperar os dados completos de uma entidade.
 - **Anotação:** `@Inheritance(strategy = InheritanceType.JOINED)`
- **Table per Concrete Class** (Uma Tabela por Classe Concreta): cada classe concreta da hierarquia é mapeada para sua própria tabela, que contém todos os atributos, incluindo os herdados da superclasse. Não há tabela para a superclasse abstrata.
 - **Anotação:** `@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)`

Mapeamento de Coleções

O **mapeamento de coleções** permite **representar e persistir associações entre entidades utilizando estruturas como listas, conjuntos, mapas e arrays**. Esses mapeamentos são úteis para gerenciar relacionamentos do tipo One-to-Many, Many-to-Many e para armazenar coleções de elementos embutidos (embeddable) ou valores básicos. O Hibernate fornece suporte abrangente para diferentes tipos de coleções, permitindo um controle detalhado sobre como essas estruturas são armazenadas e recuperadas no banco de dados.

O Hibernate suporta os seguintes tipos de coleções:

- **List** (`java.util.List`): Mantém a ordem dos elementos e permite duplicatas.

- Set (`java.util.Set`): Garante que não haja duplicatas.
- Map (`java.util.Map`): Armazena pares chave-valor.
- Array (`java.util.Array` ou tipo primitivo): Representa um array fixo.
- Collection (`java.util.Collection`): Interface genérica para coleções.

Chaves Compostas

Chaves compostas representam uma **combinação de dois ou mais campos** que, juntos, **formam uma chave primária única** para identificar uma entidade no banco de dados. Essas chaves são comuns em tabelas que participam de relacionamentos complexos ou em cenários onde nenhuma única coluna é suficiente para garantir a unicidade.

O Hibernate oferece duas abordagens principais para mapear chaves compostas:

- **Classe Embutida** (`@Embeddable`)
- **Classe de Identidade Separada** (`@IdClass`)

Na abordagem `@Embeddable`, criamos uma classe separada que encapsula os atributos da chave composta. Essa classe é anotada com `@Embeddable`, e sua instância é usada como identificador da entidade principal. As vantagens dessa abordagem incluem:

- Centralização: A chave composta é encapsulada em uma única classe, tornando o código mais limpo.
- Reutilização: A mesma classe pode ser usada em diferentes entidades.
- Consistência: Métodos `equals` e `hashCode` garantem o correto comportamento de comparação.

Já na abordagem do `@IdClass`, usamos uma classe de chave composta separada que é anotada com `@IdClass`. Essa classe define os campos que compõem a chave primária, mas os atributos são mapeados diretamente na entidade principal. Essa abordagem pode ser usada por ser:

- Simples de Mapear: Os atributos são definidos diretamente na entidade principal.
- Compatível com Estruturas Legadas: Útil para bancos de dados existentes que já utilizam chaves compostas.

Padrões de Projeto no Hibernate

O Hibernate adota e implementa vários padrões de projeto para oferecer uma solução robusta, extensível e eficiente para o mapeamento objeto-relacional (ORM). Esses padrões desempenham um papel crucial na arquitetura do Hibernate, permitindo que ele gerencie o ciclo de vida das entidades, otimize o acesso ao banco de dados e forneça flexibilidade para diferentes cenários de persistência.

Vamos focar em 4 padrões – lembrando que nosso objetivo aqui é entender como o Hibernate implementa o padrão, não explicarmos o padrão em si:

- **Repository**
- **Unit of Work**
- **Singleton**
- **Data Access Object (DAO)**

Padrão Data Access Object

O **DAO**, ou **Data Access Object**, separa a lógica de persistência da lógica de negócios, criando uma camada intermediária que encapsula o acesso ao banco de dados. Essa abordagem torna o sistema mais modular, reutilizável e fácil de manter, ao mesmo tempo que promove o princípio de **separação de responsabilidades**.

O objeto DAO é responsável por realizar as operações de acesso e manipulação dos dados no banco de dados, como **CRUD (Create, Read, Update, Delete)** e consultas personalizadas. Ele abstrai os detalhes do ORM (no caso, o Hibernate) e oferece uma interface simplificada para a camada de negócios.

A estrutura típica de um DAO no Hibernate inclui:

- Uma **interface** que define os métodos.
- Uma **implementação concreta** que utiliza o Hibernate para realizar as operações.

Padrão Repository

O **padrão Repository** é uma extensão ou especialização do padrão DAO, mas com uma abordagem mais orientada a abstração de coleções de objetos no domínio da aplicação. Ele fornece uma interface para o acesso a objetos, simulando um repositório em memória, enquanto oculta os detalhes de persistência. No Hibernate, o uso do padrão Repository é uma prática comum em projetos modernos para manter a lógica de acesso ao banco desacoplada da lógica de negócio.

Na sua implementação, ele:

- Atua como uma ponte entre a lógica de negócios e a persistência de dados.
- Simula um repositório de objetos, encapsulando as operações de acesso e manipulação no banco de dados.
- Oferece uma interface rica e focada no domínio da aplicação, enquanto esconde a complexidade do ORM.

Assim, em vez de expor diretamente consultas SQL ou HQL para as camadas superiores, o Repository oferece métodos que representam ações de alto nível, como `findById`, `save`, `delete`, ou até métodos mais específicos como `findByStatus`. Usando o framework Spring, podemos implementar o Spring Data, que é uma abordagem robusta ao padrão Repository.

Unity of Work

O **padrão Unit of Work** (Unidade de Trabalho) é um conceito importante no desenvolvimento de software, especialmente em aplicações que utilizam frameworks ORM, como o Hibernate. Ele tem como **objetivo principal gerenciar as alterações realizadas em um conjunto de objetos** em uma única operação transacional. O padrão ajuda a garantir que todas as alterações sejam aplicadas de maneira atômica, consistente e eficiente no banco de dados.

O padrão Unit of Work atua como:

- **Gerenciador de Mudanças:** Ele rastreia quais objetos foram carregados, modificados, adicionados ou removidos durante uma transação.
- **Controle Transacional:** Todas as operações realizadas no escopo do Unit of Work são tratadas como uma única transação. Se algo falhar, a operação inteira pode ser revertida.
- **Otimização de Operações:** Combina múltiplas alterações em comandos SQL eficientes para reduzir o número de interações com o banco de dados.

No contexto do Hibernate, a `Session` atua como a implementação do Unit of Work. Ela monitora entidades, rastreando o seu estado, sincroniza os dados com o banco de dados, aplicando todas as transformações no momento de uso do método `flush()`, e gerencia transações através da API de transações.

Aqui temos alguns métodos importantes, que você precisa saber:

Método	Descrição
<code>save</code> e <code>saveOrUpdate</code>	Usados para inserir ou atualizar entidades
<code>update</code>	Atualiza uma entidade existente
<code>delete</code>	Marca uma entidade existente para deleção
<code>flush</code>	Sincroniza as alterações feitas com o banco de dados, sem confirmar a transação

clear

Limpa o contexto da persistência, descartando as entidades rastreadas pela Session

Padrão Singleton

O padrão Singleton é amplamente utilizado no contexto do Hibernate para **gerenciar instâncias do SessionFactory**, que é um componente central na arquitetura do framework. O objetivo principal do Singleton é garantir que apenas uma instância do SessionFactory seja criada durante o ciclo de vida da aplicação, evitando a criação redundante de objetos caros e promovendo eficiência no uso de recursos.

O SessionFactory é um objeto pesado, que encapsula as configurações do Hibernate e o mapeamento das entidades com o banco de dados. Ele é projetado para ser *thread-safe* e reutilizado em toda a aplicação. Criar múltiplas instâncias do SessionFactory é uma prática ineficiente e desnecessária, pois cada instância consome uma quantidade significativa de memória e recursos.

DDL

No Hibernate, podemos exportar as configurações DDL (*Data Definition Language* – que comporta a definição dos objetos do modelo relacional). Isso é feito através da funcionalidade `hibernate.hbm2ddl`. Ele facilita a integração entre o modelo de dados da aplicação e o esquema do banco de dados.

Com o `hbm2ddl`, o Hibernate automatiza operações no banco de dados com base no mapeamento das entidades. Essa automação pode variar desde validação do esquema até a recriação completa das tabelas, dependendo das configurações escolhidas. Suas configurações sevem para:

- Sincronizar o modelo de dados definido nas entidades com o esquema do banco.
- Automatizar mudanças no esquema durante o ciclo de desenvolvimento.
- Validar o modelo de dados em relação ao banco em ambientes de produção.

A principal propriedade do `hbm2ddl` é o `auto` – ou `hibernate.hbm2ddl.auto`. Ele define o comportamento do Hibernate em relação ao esquema do banco, como validação, criação ou atualização, numa interação automática. Exemplos de valores que podem ser setados nesse parâmetro são:

- **validate:** Valida o esquema do banco de dados em relação ao mapeamento definido nas entidades.
- **update:** Atualiza o esquema do banco de dados para refletir as alterações no mapeamento das entidades.
- **create:** Cria o esquema do banco de dados do zero com base no mapeamento das entidades.
- **create-drop:** Similar ao create, mas também remove o esquema do banco de dados ao encerrar a sessão.
- **drop:** Irá realizar o descarte do banco de dados ao encerrar a sessão.
- **none:** Não realiza nenhuma operação no esquema do banco de dados.



(COMPERVE/UFRN/2023) O framework Java Hibernate para mapeamento Objeto-Relacional que implementa a JPA (Java Persistence API) é amplamente utilizado. A propriedade `hibernate.hbm2ddl.auto` pode ser configurada para automatizar a execução de linguagem DDL no banco de dados. No que diz respeito aos possíveis valores de `hibernate.hbm2ddl.auto`, se for atribuído o valor

- a) `drop`, o banco de dados será apagado e não será recriado.
- b) `none`, o banco de dados será validado e não será recriado.
- c) `create`, o banco de dados será criado e não será apagado.
- d) `alter`, o banco de dados será atualizado e não será apagado.

Comentários:

Vamos analisar os itens.

- a) Certo. Com o parâmetro `drop` o banco de dados é apagado e não será recriado, sendo descartado.
- b) Errado. O parâmetro `None` não faz nada com o esquema.
- c) Errado. O `Create` cria um novo esquema, "destruindo" os esquemas anteriores existentes.
- d) Errado. O `Alter` irá atualizar o esquema, não o banco de dados.

Portanto, correta a letra A.

Estratégias de Bloqueio

Concorrência é um dos problemas em modelos de dados. No Hibernate, dentro das estratégias de bloqueio (*locks*), temos duas abordagens: as **estratégias otimistas** e as **estratégias pessimistas**. Essas estratégias são utilizadas para gerenciar o acesso simultâneo a dados compartilhados em um banco de dados, garantindo a consistência e integridade dos dados. Ambas as estratégias são abordagens para lidar com problemas de concorrência, mas com filosofias e comportamentos diferentes.

A **concorrência otimista** assume que **conflitos entre transações simultâneas são raros**. Essa abordagem permite que várias transações acessem e modifiquem os mesmos dados simultaneamente, mas verifica conflitos no momento da **confirmação (commit)**.

Quando uma transação lê uma entidade, ela armazena um estado adicional para verificar alterações, geralmente uma versão ou timestamp. transação modifica a entidade sem bloqueá-la no banco de dados. Antes de aplicar as alterações ao banco, verifica-se se o estado atual do dado no banco é o mesmo que foi carregado originalmente pela transação. Se houve modificações por outra transação, um conflito é detectado, e uma exceção é lançada.

Já na **concorrência pessimista** assume que conflitos entre transações são prováveis. Essa abordagem evita conflitos bloqueando os dados no momento em que são acessados, garantindo que nenhuma outra transação possa modificá-los até que o bloqueio seja liberado.

O funcionamento é mais ou menos o seguinte: quando uma transação lê ou modifica uma entidade, ela bloqueia o registro no banco de dados. Temos dois estados principais de bloqueio:

- Shared Lock (Leitura): Permite leitura por múltiplas transações, mas impede alterações.
- Exclusive Lock (Escrita): Impede tanto leitura quanto escrita por outras transações.

Esse bloqueio fica mantido até que a transação seja concluída, seja por commit, seja por rollback.

Aspecto	Otimista	Pessimista
Abordagem	Detecta conflitos no commit.	Previne conflitos com bloqueios.
Bloqueio	Não utiliza bloqueio no banco.	Usa bloqueios no banco de dados.
Uso Ideal	Sistemas com poucas atualizações.	Sistemas com alta concorrência.
Desempenho	Melhor em leitura intensiva.	Pode impactar a escalabilidade.
Conflitos	Detectados tardiamente.	Prevenidos de forma antecipada.
Risco de Deadlocks	Não há risco.	Deadlocks podem ocorrer.



(CEBRASPE/SERPRO/2023) Julgue o próximo item, relativo a tecnologias backend.

Em relação à gestão de bloqueio em um banco de dados relacional, o Hibernate 6.2 utiliza a estratégia otimista, uma vez que essa estratégia assume que várias transações podem ser concluídas sem afetar outras, logo, nessa versão, prescinde-se da estratégia pessimista.

Comentários:

O Hibernate permite usar tanto uma abordagem otimista, quanto pessimista. Além disso, a abordagem otimista pressupõe que conflitos serão raros – e, quando ocorrerem, serão tratados por alguma exception. (Gabarito: Errado)

Resumo de Anotações

Vamos reunir aqui as principais anotações do Hibernate. São várias, e muitas delas podem cair em prova, então recomendo a leitura atenta.



ACORDE!

Anotação	Descrição
@Entity	Marca uma classe como uma entidade que será mapeada para uma tabela no banco de dados.
@Table	Especifica o nome da tabela e outras informações de mapeamento, como esquema e índices.
@Id	Indica o campo que será usado como chave primária da entidade.
@GeneratedValue	Configura como os valores da chave primária serão gerados (auto, identidade, sequência, etc.).
@Column	Personaliza o mapeamento de um campo para uma coluna, permitindo definir nome, tamanho, nulidade e outras propriedades.
@OneToOne	Define um relacionamento de um para um entre duas entidades.
@OneToMany	Define um relacionamento de um para muitos entre duas entidades.
@ManyToOne	Define um relacionamento de muitos para um entre duas entidades.
@ManyToMany	Define um relacionamento de muitos para muitos entre duas entidades.
@JoinColumn	Especifica a coluna de junção em um relacionamento entre entidades.
@JoinTable	Configura a tabela intermediária usada em relacionamentos @ManyToMany.
@Embedded	Indica que uma classe embutida será usada como parte da entidade.
@Embeddable	Define uma classe que será incorporada em outra entidade como uma composição.

@Transient	Indica que um campo não será persistido no banco de dados.
@Lob	Marca um campo como um objeto grande (LOB), geralmente usado para textos longos ou binários.
@Inheritance	Configura o tipo de estratégia de herança para uma hierarquia de classes (SINGLE_TABLE, JOINED, TABLE_PER_CLASS).
@DiscriminatorColumn	Define a coluna usada para discriminar classes em uma estratégia de herança do tipo SINGLE_TABLE.
@SequenceGenerator	Configura um gerador de sequência usado para criar valores de chaves primárias automaticamente.
@NamedQuery	Define uma consulta HQL estática e reutilizável associada à entidade.
@NamedNativeQuery	Define uma consulta SQL nativa estática e reutilizável associada à entidade.
@ElementCollection	Configura coleções de tipos básicos ou objetos embutidos em uma entidade.
@Fetch	Configura o comportamento de carregamento (FetchType.LAZY ou FetchType.EAGER) em relacionamentos.
@Cascade	Configura operações em cascata (persistência, remoção, etc.) para os relacionamentos.
@Version	Marca um campo para controle de versão da entidade, usado em concorrência otimista.
@OptimisticLock	Define estratégias de controle de concorrência para gerenciamento de atualizações de registros.
@Cache	Configura opções de cache de segundo nível para uma entidade ou coleção.
@OrderBy	Especifica a ordenação dos elementos em uma coleção ao serem carregados do banco.
@OrderColumn	Configura a coluna que será usada para manter a ordem dos elementos em uma coleção.
@NaturalId	Define um identificador natural que pode ser usado como chave única alternativa ao @Id.

QUESTÕES COMENTADAS

01. (IBFC/TRF 5/2024) Considerando as afirmativas abaixo sobre o framework Hibernate, dê valores Verdadeiro (V) ou Falso (F).

- () O Hibernate é exclusivamente utilizado para a criação de interfaces gráficas em aplicações Java.
- () A utilização do Hibernate elimina a necessidade de conhecimento sobre a linguagem SQL, já que todas as operações de banco de dados são realizadas automaticamente pelo framework.
- () O Hibernate é limitado a bancos de dados específicos e não oferece suporte a uma variedade de provedores de banco de dados relacionais.

Assinale a alternativa que apresenta a sequência correta de cima para baixo.

- a) F - F - F
- b) V - F - V
- c) F - V - F
- d) V - V - V

Comentários:

Vamos analisar os itens.

() *O Hibernate é exclusivamente utilizado para a criação de interfaces gráficas em aplicações Java.*
Falso. O Hibernate é um framework de ORM, ele não faz interfaces gráficas.

() *A utilização do Hibernate elimina a necessidade de conhecimento sobre a linguagem SQL, já que todas as operações de banco de dados são realizadas automaticamente pelo framework.*
Falso. Ainda que o Hibernate automatize muitos processos, precisamos implementar alguma forma de linguagem SQL – seja o SQL puro, JPQL ou HQL.

() *O Hibernate é limitado a bancos de dados específicos e não oferece suporte a uma variedade de provedores de banco de dados relacionais.*
Falso. O Hibernate suporta diversos bancos de dados relacionais.

Portanto, todos os itens são falsos.

Gabarito: Letra A

02. (IBFC/TRF 5/2024) O sistema _____ é muito usado para o mapeamento de objetos Java para relações, ele oferece uma implementação do Java Persistence API (JPA). Assinale a alternativa que

preencha corretamente a lacuna, tendo como tema a arquitetura de soluções em banco de dados para Web.

- a) Django
- b) Hibernate
- c) Alchemy
- d) Json

Comentários:

Vamos analisar as alternativas.

- a) Errado. Django é um framework web em Python.
- b) Certo. O Hibernate implementa ORM a partir da JPA, sendo nossa resposta.
- c) Errado. Alchemy é um termo amplo, podendo se referir a diversas ferramentas.
- d) Certo. JSON é um formato de arquivos de dados.

Portanto, correta a letra B.

Gabarito: Letra B

03. (FGV/PGM Niterói/2023) No contexto do Hibernate, considere a declaração a seguir.

```
Estudante estudante= new Estudante();
```

É correto afirmar que o objeto estudante, ao longo da sua existência/utilização, pode estar em algum dos estados:

- a) Defined, Instanced e Latent;
- b) Lazy, Eager e Restfull;
- c) On, Loaded e Off;
- d) Open, Active e Closed;
- e) Transient, Persistent e Detached.

Comentários:

Uma entidade, no Hibernate, pode estar em três estados: Transient, Persistent e Detached.

Gabarito: Letra E

04. (CEBRASPE/TBG/2023)

Data Access Layer	
Java Persistence API	Hibernate Native API
JDBC	
Relational Database	
Hibernate	

Com base na hierarquia precedente, julgue o item seguinte, a respeito de Hibernate.

Considerado o conceito da solução ORM Hibernate, conclui-se que a hierarquia apresentada anteriormente está errada. O correto seria a hierarquia que se segue, uma vez que o Hibernate não se incorpora ao banco de dados relacional, mas se encontra entre a camada de acesso a dados do aplicativo Java e o banco de dados relacional.

Data Access Layer	
Java Persistence API	Hibernate Native API
Hibernate	
JDBC	
Relational Database	

Comentários:

O item está correto. O Hibernate faz parte da camada de acesso a dados, estando logicamente acima do JDBC, uma vez que ele abstrai a complexidade das operações de baixo nível realizadas por meio do JDBC. Ele transforma os objetos Java em registros do banco de dados relacional e vice-versa. Por isso, o Hibernate não é uma camada que se incorpora diretamente ao banco de dados, mas sim uma ferramenta que ajuda na integração entre as classes Java e o banco de dados relacional.

Gabarito: Certo

05. (CEBRASPE/TC DF/2023) Julgue o próximo item, relativo a desenvolvimento web em Java e a desenvolvimento com contêiner.

O framework Hibernate gera o SQL para determinado banco de dados no momento da compilação do código fonte.

Comentários:

O Hibernate, como framework ORM (Object-Relational Mapping), não gera o SQL no momento da compilação do código fonte. Em vez disso, o Hibernate utiliza o mapeamento configurado (via arquivos XML ou anotações nas classes Java) para gerar dinamicamente as instruções SQL em

tempo de execução, conforme as operações realizadas na aplicação (por exemplo, consultas, inserções, atualizações ou exclusões).

Gabarito: Errado

06. (COMPERVE/UFRN/2023) Ao utilizar o framework Hibernate no mapeamento objeto-relacional de uma aplicação, uma entidade pode assumir diferentes estados de acordo com a sua vinculação à Hibernate Session. Sobre a mudança de estado desses objetos nesse contexto, um objeto no estado

- a) transient muda para persistent ao ser associado com uma Hibernate Session.
- b) persistent muda para transient quando a Hibernate Session é fechada.
- c) transient muda para detach ao ser desassociado de uma Hibernate Session.
- d) detach muda para persistent quando a Hibernate Session é fechada.

Comentários:

A questão aborda as mudanças de estados das entidades no contexto do Hibernate. Vamos analisar as alternativas e encontrar a correta.

- a) Certo. Ao alocarmos uma Session a uma entidade transiente, ela passa a ser monitorada e se torna persistente.
- b) Errado. Ao fecharmos a Session, uma entidade persistente se torna detached.
- c) Errado. Novamente, ao desassociarmos uma entidade de uma Session, ela vai de Persistent para Detached.
- d) Errado. Uma outra vez, temos a entidade indo de Persistent para Detached quando a Session é fechada.

Portanto, correta a letra A. Ainda temos o estado de remoção, que não foi abordado pela questão.

Gabarito: Letra A

07. (VUNESP/DPE SP/2023) Em uma aplicação Java, foi encontrado o seguinte trecho de conteúdo no arquivo de mapeamento do framework Hibernate.

```
<hibernate-mapping>
  <class name= Funcionario table= Func >
    <id name= id type= int column= id >
      <generator class= native />
    </id>
    <property name= nome column= first_name
      type= string />
    <property name= sobrenome
```

```
        column= last_name  
        type= string />  
    <property name= salario column= salary  
        type= int />  
</class>  
</hibernate-mapping>
```

Assinale a alternativa correta a respeito.

- a) A classe Java Funcionario corresponde a um servlet (é derivada de HttpServlet), provendo chamadas para consultar e armazenar registros de funcionários em um banco de dados.
- b) O schema do banco de dados utilizado deve possuir uma tabela chamada Funcionario para que instâncias da classe persistente de mesmo nome sejam armazenadas na forma de registros.
- c) O Hibernate escolherá os algoritmos identity, sequence ou hilo para criar a chave primária na tabela de funcionários, dependendo das capacidades do servidor de banco de dados utilizado.
- d) O fato de esse arquivo de mapeamento estar escrito em representação XML implica que o banco de dados subjacente deve ser do tipo Document-Based, ou seja, da categoria NoSQL.
- e) A classe Func da aplicação Java está mapeada na tabela Func de um banco de dados relacional que provê persistência para a aplicação.

Comentários:

Vamos analisar cada alternativa.

- a) Errado. A classe Funcionario no mapeamento é uma classe de modelo, usada pelo Hibernate para representar entidades persistentes no banco de dados. Não há nenhuma relação com HttpServlet ou comportamento de um servlet. O Hibernate lida apenas com persistência e não com a camada de controle típica de servlets.
- b) Errado. O mapeamento especifica que a tabela correspondente à classe Funcionario é chamada Func (conforme indicado no atributo table="Func"). Portanto, o banco de dados deve possuir uma tabela chamada Func, e não Funcionario.
- c) Certo. O atributo `<generator class="native" />` indica que o Hibernate usará a estratégia "native" para geração de chaves primárias, que delega ao banco de dados o uso do algoritmo apropriado, como identity, sequence ou hilo, dependendo das capacidades específicas do banco configurado.
- d) Errado. O uso de XML no mapeamento é uma característica do Hibernate para configurar a persistência em bancos de dados relacionais (RDBMS). Não há qualquer relação com bancos de dados NoSQL ou Document-Based. A configuração no XML mapeia classes Java para tabelas relacionais.

- e) Errado. A alternativa menciona "classe Func", enquanto no mapeamento apresentado, a classe Java referenciada no atributo class name="Funcionario" é Funcionario, e não Func. O nome Func refere-se apenas à tabela do banco de dados (definido no atributo table="Func"), e não à classe Java.

Sendo assim, correta a letra C.

Gabarito: Letra C

08. (FGV/ALEMA/2023) O Hibernate 6.1.x é uma ferramenta ORM; a interface que permite executar consultas SQL em banco de dados se chama

- a) JNI.
- b) JDBC.
- c) QueryHint.
- d) NativeQuery.
- e) ConnectionsProvider.

Comentários:

Para implementarmos consultas nativas com SQL usamos o NativeQuery.

Gabarito: Letra D

09. (FGV/Câmara dos Deputados/2023) No âmbito da arquitetura do Hibernate, assinale o objeto que estabelece uma conexão física com um banco de dados e é projetado para ser instanciado a cada vez que seja necessária uma interação com o mesmo.

- a) Criteria object.
- b) Query object.
- c) Session object.
- d) SessionFactory object.
- e) Transaction object.

Comentários:

Quando começamos a monitorar uma entidade no Hibernate, estabelecendo uma conexão com o banco de dados e "sincronizando" as informações, usamos um objeto do tipo Session.

Gabarito: Letra C

10. (COMPERVE/UFRN/2023) O framework Java Hibernate para mapeamento Objeto-Relacional que implementa a JPA (Java Persistence API) é amplamente utilizado. A propriedade `hibernate.hbm2ddl.auto` pode ser configurada para automatizar a execução de linguagem DDL no banco de dados. No que diz respeito aos possíveis valores de `hibernate.hbm2ddl.auto`, se for atribuído o valor

- a) `drop`, o banco de dados será apagado e não será recriado.
- b) `none`, o banco de dados será validado e não será recriado.
- c) `create`, o banco de dados será criado e não será apagado.
- d) `alter`, o banco de dados será atualizado e não será apagado.

Comentários:

Vamos analisar os itens.

- a) Certo. Com o parâmetro `drop` o banco de dados é apagado e não será recriado, sendo descartado.
- b) Errado. O parâmetro `None` não faz nada com o esquema.
- c) Errado. O `Create` cria um novo esquema, "destruindo" os esquemas anteriores existentes.
- d) Errado. O `Alter` irá atualizar o esquema, não o banco de dados.

Portanto, correta a letra A.

Gabarito: Letra A

11. (VUNESP/CM SOB/2023) No framework Hibernate do Java, o estado conhecido como `desanexado` (`detached`) ocorre quando

- a) o garbage collector da JVM detecta um objeto sem referência, mas ainda não o destruiu de fato.
- b) um objeto de uma classe persistente é associado a uma sessão e adquire uma representação no banco de dados, e também um valor de identificador.
- c) um objeto persistente acabou de ser destruído e sua representação no banco de dados foi excluída, sendo que a referência ao objeto não é mais válida.
- d) um objeto de uma classe persistente acabou de ser instanciado, mas ainda não está associado a uma sessão Hibernate.
- e) um objeto já esteve no estado persistente, mas teve sua sessão fechada, sendo que a referência ao objeto ainda é válida.

Comentários:

Um objeto no Hibernate se torna detached quando esse objeto já foi persistente, e teve sua sessão encerrada, sem remoção do objeto do contexto do banco de dados. Nesse sentido, a alternativa que melhor explica essa interação é a letra E.

Gabarito: Letra E

12. (CEBRASPE/AGER MT/2023) Assinale a opção que indica corretamente o exemplo de framework de persistência de dados em que há o mapeamento entre uma representação do modelo de objeto para uma representação do modelo de dados relacional.

- a) hibernate
- b) spring boot
- c) ESB (enterprise service bus)
- d) RMI (remote method invocation)
- e) MVC (model-view-controller)

Comentários:

Das alternativas apresentadas, a única que tem um framework para a persistência de dados, que implementa o JPA, é o Hibernate.

Gabarito: Letra A

13. (VUNESP/Pref. Pindamonhangaba/2023) No framework Hibernate do Java, o estado de um objeto de uma classe persistente que acabou de ser instanciado, mas que ainda não está associado a uma sessão, não possui representação no banco de dados e também não possui um identificador atribuído é conhecido como

- a) inicial.
- b) transiente.
- c) persistente.
- d) desanexado (detached).
- e) flutuante.

Comentários:

Lembrem-se do ciclo de vida padrão das entidades: Transient → Persistent → Detached. Então, quando temos uma entidade que não possui representação e ainda não é acompanhada por uma Session, temos uma entidade no estado Transient, ou transiente.

Gabarito: Letra B

14. (FGV/ALEMA/2023) Analise o código a seguir, escrito em linguagem Java e com anotações do Hibernate 6.1. Considere que os métodos de acesso get e set foram omitidos da especificação das classes abaixo para efeitos de brevidade.

```
1 @MappedSuperclass
2 public static class Pessoa {
3
4     @Id
5     private Long id;
6
7     private String nome;
8
9     @Column(length = 11)
10    private String cpf;
11 }
12
13 @Entity(name = "Professor")
14 public static class Professor extends Pessoa {
15
16     private BigDecimal salario;
17 }
18
19 @Entity(name = "Aluno")
20 public static class Aluno extends Pessoa {
21
22     private boolean diurno;
23 }
```

Sobre o modelo de banco de dados decorrente deste mapeamento objeto-relacional, é correto afirmar que

- a) o atributo nome da tabela Pessoa será do tipo VARCHAR.
- b) o atributo salario da tabela Professor não poderá ser nulo.
- c) o atributo id da tabela Aluno será uma chave primária.
- d) o atributo diurno da tabela Pessoa poderá ter só 2 valores.
- e) o atributo cpf da tabela Pessoa poderá ter 11 caracteres.

Comentários:

Vamos analisar as afirmativas.

- a) Errado. Não temos menção expressa no código dessa propriedade. Precisaríamos ter o VARCHAR indicado junto do atributo nome, na classe Pessoa.
- b) Errado. No código, não há nenhuma anotação indicando que o atributo salario é obrigatório, como @NotNull. Portanto, o atributo salario poderá ser nulo no banco de dados.

- c) Certo. A classe Aluno herda de Pessoa, que está anotada como @MappedSuperclass, e possui o atributo id anotado com @Id. Isso significa que o atributo id será herdado pela tabela correspondente a Aluno e será configurado como chave primária.
- d) Errado. O atributo diurno pertence à classe Aluno, não à classe Pessoa. Além disso, o atributo é do tipo boolean, o que permitirá apenas dois valores (true ou false), mas isso não é aplicável à tabela Pessoa, pois o atributo não está presente nela.
- e) Errado. A especificação de tamanho é para a coluna, não para o CPF.

Gabarito: Letra C

15. (CEBRASPE/SERPRO/2023) Julgue o próximo item, relativo a tecnologias backend.

Em relação à gestão de bloqueio em um banco de dados relacional, o Hibernate 6.2 utiliza a estratégia otimista, uma vez que essa estratégia assume que várias transações podem ser concluídas sem afetar outras, logo, nessa versão, prescinde-se da estratégia pessimista.

Comentários:

O Hibernate permite usar tanto uma abordagem otimista, quanto pessimista. Além disso, a abordagem otimista pressupõe que conflitos serão raros – e, quando ocorrerem, serão tratados por alguma exception.

▪

Gabarito: Errado

16. (FGV/SEFAZ AM/2022) O Hibernate, versão 6, é um framework que oferece mecanismos para armazenamento e recuperação de objetos Java por meio de mapeamento objeto-relacional.

Em relação a interface Session, que oferece operações sobre instâncias de classes de entidade mapeadas, assinale a afirmativa correta.

- a) Uma instância transiente pode se tornar persistente chamando o método detach (Object object).
- b) Qualquer instância retornada pelo método get (String entityName, Object id) é persistente.
- c) Uma instância persistente pode se tornar transiente por meio do método merge (T object).
- d) Todas as instâncias persistentes são marcadas para remoção chamando o método flush().
- e) O estado original das instâncias gerenciadas é carregado na memória chamando o método reload().

Comentários:

Vamos analisar as alternativas.

- a) Errado. O método `detach(Object object)` é usado para desanexar uma entidade gerenciada pelo contexto de persistência, tornando-a uma instância desanexada (`detached`). Ele não converte uma instância transiente (que nunca foi persistida) em persistente.
- b) Certo. O método `get(String entityName, Object id)` retorna uma instância da entidade identificada pelo nome (`entityName`) e pelo identificador (`id`). Se o objeto for encontrado no banco de dados, ele será retornado como uma instância persistente, ou seja, gerenciada pelo contexto de persistência do Hibernate.
- c) Errado. O método `merge(T object)` não torna uma instância persistente em transiente. Pelo contrário, ele mescla o estado de uma instância desanexada (`detached`) ou de uma nova instância no contexto de persistência, criando ou atualizando o estado correspondente no banco de dados. Isso torna a entidade persistente.
- d) Errado. O método `flush()` sincroniza o estado das entidades gerenciadas pelo contexto de persistência com o banco de dados, mas não marca as entidades como removidas. A marcação para remoção é feita explicitamente pelo método `remove()`.
- e) Errado. Não existe um método chamado `reload()` na interface `Session` do Hibernate. Para recarregar uma entidade gerenciada pelo contexto, usamos o método `refresh(Object object)`.

Portanto, correta a letra B.

Gabarito: Letra B

17. (CEBRASPE/PC PB/2022) Em desenvolvimento web, é comum a utilização de classes, tecnologias ou mesmo ferramentas e frameworks, como aquela que utiliza sua própria linguagem de consulta, o que facilita a etapa de desenvolvimento quanto ao acesso a bancos de dados e SQL. Esse exemplo enfatiza uma das vantagens do

- a) asynchronous javascript and XML.
- b) servlet.
- c) hibernate.
- d) javaserver faces.
- e) javaserver pages.

Comentários:

A questão aborda o uso do *mapeamento objeto-relacional*, que facilita a implementação e persistência e consultas SQL em bancos de dados a partir de classes e objetos. Das alternativas apresentadas, aquela que apresenta um framework de persistência é a letra C, Hibernate.

Gabarito: Letra C

18. (FCC/TRT 22/2022) Uma analista está desenvolvendo um sistema usando JPA e deseja configurar a utilização do framework Hibernate como provedor de persistência, por meio da instrução `<provider>org.hibernate.ejb.HibernatePersistence</provider>` que deve ser colocada no arquivo

- a) hibernate.xml
- b) persistence-unit.xml
- c) persistence.xml
- d) orm-database-config.xml
- e) server.xml

Comentários:

A configuração do Hibernate é feita num arquivo denominado `persistence.xml`.

Gabarito: Letra C

19. (CEBRSAPE/FUB/2022) Julgue o item a seguir, a respeito de projeto e desenvolvimento de sistemas.

Hibernate é um modelo computacional que faz parte de um processo avançado de gerenciamento de memória dos protocolos comumente utilizados em redes legadas.

Comentários:

O Hibernate é um framework de ORM em Java – não tem relação com memória de protocolos.

Gabarito: Errado

20. (FCC/TRT 19/2022) No Hibernate uma entity bean deve ter uma chave primária, que é indicada por meio da anotação

- a) `@PrimaryKey`
- b) `@PrymaryId`
- c) `@Id`
- d) `@SpecialKey`
- e) `@PK`

Comentários:

Questão tranquila. Para identificarmos um atributo como chave primária, usamos a anotação `@Id`.

21. (FCC/TRT 14/2022) Considere o método abaixo, presente em uma aplicação que utiliza Hibernate 4.3 original, executado em condições ideais.

```
private void createAndStoreEvent(Processo p) {  
    Session session = HibernateUtil.getSessionFactory().getCurrentSession();  
    session.beginTransaction();  
    ____ ;  
    session.getTransaction().commit();  
}
```

Para salvar o objeto p na tabela do banco de dados utiliza-se, na lacuna I, a instrução

- a) session.save(p)
- b) session.deploy(p)
- c) session.add(p)
- d) session.persist(p)
- e) session.send(p)

Comentários:

O método para persistirmos um objeto em uma tabela, salvando-o, é o save(). No caso, usaremos session.save(p) para salvar na tabela Processo. Atenção para o método persist(), que permite reanexarmos um objeto a uma sessão, que passará a fazer as persistências sem necessidade de save().

22. (FGV/CGU/2022) Considere o desenvolvimento de uma aplicação em um contexto de programação concorrente, em que é esperada a execução simultânea de múltiplas tarefas computacionais. O time utiliza a linguagem de programação JAVA e o framework ORM Hibernate no projeto. Para obter um objeto do tipo SessionFactory, o time decide utilizar o arquivo de configuração e as seguintes classes:

```
org.hibernate.cfg.Configuration,  
org.hibernate.service.ServiceRegistry e  
org.hibernate.service.ServiceRegistryBuilder.
```

No início do desenvolvimento da primeira funcionalidade com linhas de execução paralelas (Threads), o time discute se é seguro compartilhar o objeto do tipo SessionFactory.

Um dos desenvolvedores afirma que é seguro.

O time concluiu acertadamente que esse desenvolvedor está:

- a) errado, pois a classe SessionFactoryImpl implementa a interface Referenceable;
- b) errado, pois a classe SessionFactoryImpl não é thread-safe;
- c) errado, pois a classe SessionFactoryImplementor não é thread-safe;
- d) correto, pois o objeto do tipo SessionFactory é imutável e seu estado interno contém os metadados ORM;
- e) correto, pois a classe SessionFactoryImplementor implementa a interface ConcurrentHashMap.

Comentários:

A classe SessionFactory é uma classe que carrega os metadados do ORM, além de ser Thread-Safe. Então, é possível o compartilhamento dela, já que ela é thread-safe e imutável.

Gabarito: Letra D

23. (CC TJAC/TJ AC/2022) Hibernate é o framework para persistência de dados utilizado em projetos Java. Sendo uma das opções a implementar o conceito de mapeamento objeto-relacional (ORM). Avalie as asserções a seguir e a relação proposta entre elas

I - Com a utilização do Hibernate, não há necessidade de escrever SQL "puro", pois ele utiliza seu próprio código, chamado de HQL (Hibernate Query Language)

PORQUE

II - O Hibernate transforma os dados da estrutura lógica de um banco de dados em objetos relacionais.

A respeito dessas asserções, assinale a opção correta.

- a) As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.

Comentários:

Analisando a asserção 1, podemos afirmar que ela é correta – o Hibernate retira a necessidade de SQL puro, ou de criar operações CRUD tradicionais. Isso pois ele transforma os objetos em estruturas relacionais, através do ORM – configurando, assim, a asserção 2 uma explicação correta da asserção 1.

Gabarito: Letra A

24. (QUADRIX/PRODAM AM/2022) Com relação ao Hibernate, assinale a alternativa correta.

- a) O Hibernate contém diversas funcionalidades, como engenharia reversa, exportação de esquemas e anotações próprias para simplificar a utilização da API.
- b) O Hibernate é considerado um framework completo em razão de não necessitar adicionar nenhuma dependência para o seu correto funcionamento.
- c) A anotação `org.hibernate.annotations.ForeignKey` tem como função determinar o nome de uma chave primária.
- d) Uma das vantagens do Hibernate é o fato de não haver necessidade de definir identificadores para as entidades persistentes.
- e) O elemento `mapping` do arquivo `hibernate.cfg.xml` é usado para definir o usuário de conexão ao banco de dados.

Comentários:

Vamos analisar as alternativas.

- a) Certo. O Hibernate oferece diversas funcionalidades que vão além do simples mapeamento objeto-relacional (ORM). Entre essas funcionalidades estão a engenharia reversa, exportação de esquemas e anotações próprias.
- b) Errado. O Hibernate exige a inclusão de diversas dependências externas, como as relacionadas ao driver do banco de dados e a API JPA, para funcionar corretamente. Portanto, ele não é um framework "completo" no sentido de funcionar sem dependências adicionais.
- c) Errado. A anotação `org.hibernate.annotations.ForeignKey` foi removida a partir do Hibernate 5 e não deve ser usada. O nome de chaves primárias é configurado com outras anotações, como `@Id` e `@GeneratedValue`, e o nome de chaves estrangeiras pode ser definido no nível da tabela com `@JoinColumn` ou `@JoinTable`.
- d) Errado. No Hibernate, toda entidade persistente deve ter um identificador único (chave primária). Isso é obrigatório para que o framework possa gerenciar as entidades no contexto de persistência. O identificador é definido com a anotação `@Id`.
- e) Errado. O elemento `<mapping>` no arquivo `hibernate.cfg.xml` é usado para especificar os arquivos de mapeamento de entidades (como XML ou anotações). A configuração do

usuário de conexão ao banco de dados é feita por meio da propriedade `<property name="hibernate.connection.username">`.

Portanto, correta a letra A.

Gabarito: Letra A

25. (CEBRASPE/TJ PA/2020) O Hibernate tem suporte para mais de um tipo de herança. Na estratégia Tabela por Classe,

- a) várias associações unidimensionais são suportadas.
- b) há suporte às anotações para a geração de Id (IDENTITY).
- c) a classe deve ter anotações AUTO.
- d) a classe deve ser escrita sem anotações IDENTITY.
- e) as propriedades das superclasses e subclasses são mapeadas na mesma tabela.

Comentários:

Vamos analisar as alternativas.

- a) Errado. A alternativa menciona "associações unidimensionais", mas esse termo não está relacionado diretamente à estratégia Tabela por Classe. O Hibernate suporta associações bidirecionais e unidirecionais em qualquer estratégia de herança.
- b) Errado. A estratégia IDENTITY não é suportada na herança Tabela por Classe, pois o identificador precisa ser compartilhado entre as tabelas, e o IDENTITY está vinculado à geração de IDs individuais em tabelas específicas.
- c) Errado. Não há exigência de uma anotação AUTO específica para essa estratégia. A herança é configurada com `@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)`.
- d) Certo. Na estratégia Tabela por Classe, as classes não devem utilizar a estratégia IDENTITY, devendo adotar estratégias como SEQUENCE ou TABLE para garantir que os identificadores possam ser gerenciados de forma consistente.
- e) Errado. Essa descrição corresponde à estratégia SINGLE_TABLE, onde todas as propriedades das superclasses e subclasses são mapeadas na mesma tabela.

Portanto, correta a letra D.

Gabarito: Letra D

LISTA DE QUESTÕES

01. (IBFC/TRF 5/2024) Considerando as afirmativas abaixo sobre o framework Hibernate, dê valores Verdadeiro (V) ou Falso (F).

- () O Hibernate é exclusivamente utilizado para a criação de interfaces gráficas em aplicações Java.
- () A utilização do Hibernate elimina a necessidade de conhecimento sobre a linguagem SQL, já que todas as operações de banco de dados são realizadas automaticamente pelo framework.
- () O Hibernate é limitado a bancos de dados específicos e não oferece suporte a uma variedade de provedores de banco de dados relacionais.

Assinale a alternativa que apresenta a sequência correta de cima para baixo.

- a) F - F - F
- b) V - F - V
- c) F - V - F
- d) V - V - V

02. (IBFC/TRF 5/2024) O sistema _____ é muito usado para o mapeamento de objetos Java para relações, ele oferece uma implementação do Java Persistence API (JPA). Assinale a alternativa que preencha corretamente a lacuna, tendo como tema a arquitetura de soluções em banco de dados para Web.

- a) Django
- b) Hibernate
- c) Alchemy
- d) Json

03. (FGV/PGM Niterói/2023) No contexto do Hibernate, considere a declaração a seguir.

```
Estudante estudante= new Estudante();
```

É correto afirmar que o objeto estudante, ao longo da sua existência/utilização, pode estar em algum dos estados:

- a) Defined, Instanced e Latent;
- b) Lazy, Eager e Restfull;
- c) On, Loaded e Off;
- d) Open, Active e Closed;
- e) Transient, Persistent e Detached.

04. (CEBRASPE/TBG/2023)

Data Access Layer	
Java Persistence API	Hibernate Native API
JDBC	
Relational Database	
Hibernate	

Com base na hierarquia precedente, julgue o item seguinte, a respeito de Hibernate.

Considerado o conceito da solução ORM Hibernate, conclui-se que a hierarquia apresentada anteriormente está errada. O correto seria a hierarquia que se segue, uma vez que o Hibernate não se incorpora ao banco de dados relacional, mas se encontra entre a camada de acesso a dados do aplicativo Java e o banco de dados relacional.

Data Access Layer	
Java Persistence API	Hibernate Native API
Hibernate	
JDBC	
Relational Database	

05. (CEBRASPE/TC DF/2023) Julgue o próximo item, relativo a desenvolvimento web em Java e a desenvolvimento com contêiner.

O framework Hibernate gera o SQL para determinado banco de dados no momento da compilação do código fonte.

06. (COMPERVE/UFRN/2023) Ao utilizar o framework Hibernate no mapeamento objeto-relacional de uma aplicação, uma entidade pode assumir diferentes estados de acordo com a sua vinculação à Hibernate Session. Sobre a mudança de estado desses objetos nesse contexto, um objeto no estado

- a) transient muda para persistent ao ser associado com uma Hibernate Session.
- b) persistent muda para transient quando a Hibernate Session é fechada.
- c) transient muda para detach ao ser desassociado de uma Hibernate Session.
- d) detach muda para persistent quando a Hibernate Session é fechada.

07. (VUNESP/DPE SP/2023) Em uma aplicação Java, foi encontrado o seguinte trecho de conteúdo no arquivo de mapeamento do framework Hibernate.

```
<hibernate-mapping>
  <class name= Funcionario table= Func >
    <id name= id type= int column= id >
      <generator class= native />
    </id>
    <property name= nome column= first_name
      type= string />
    <property name= sobrenome
      column= last_name
      type= string />
    <property name= salario column= salary
      type= int />
  </class>
</hibernate-mapping>
```

Assinale a alternativa correta a respeito.

- a) A classe Java Funcionario corresponde a um servlet (é derivada de HttpServlet), provendo chamadas para consultar e armazenar registros de funcionários em um banco de dados.
- b) O schema do banco de dados utilizado deve possuir uma tabela chamada Funcionario para que instâncias da classe persistente de mesmo nome sejam armazenadas na forma de registros.
- c) O Hibernate escolherá os algoritmos identity, sequence ou hilo para criar a chave primária na tabela de funcionários, dependendo das capacidades do servidor de banco de dados utilizado.
- d) O fato de esse arquivo de mapeamento estar escrito em representação XML implica que o banco de dados subjacente deve ser do tipo Document-Based, ou seja, da categoria NoSQL.
- e) A classe Func da aplicação Java está mapeada na tabela Func de um banco de dados relacional que provê persistência para a aplicação.

08. (FGV/ALEMA/2023) O Hibernate 6.1.x é uma ferramenta ORM; a interface que permite executar consultas SQL em banco de dados se chama

- a) JNI.
- b) JDBC.
- c) QueryHint.
- d) NativeQuery.
- e) ConnectionsProvider.

09. (FGV/Câmara dos Deputados/2023) No âmbito da arquitetura do Hibernate, assinale o objeto que estabelece uma conexão física com um banco de dados e é projetado para ser instanciado a cada vez que seja necessária uma interação com o mesmo.

- a) Criteria object.
- b) Query object.
- c) Session object.
- d) SessionFactory object.
- e) Transaction object.

10. (COMPERVE/UFRN/2023) O framework Java Hibernate para mapeamento Objeto-Relacional que implementa a JPA (Java Persistence API) é amplamente utilizado. A propriedade `hibernate.hbm2ddl.auto` pode ser configurada para automatizar a execução de linguagem DDL no banco de dados. No que diz respeito aos possíveis valores de `hibernate.hbm2ddl.auto`, se for atribuído o valor

- a) `drop`, o banco de dados será apagado e não será recriado.
- b) `none`, o banco de dados será validado e não será recriado.
- c) `create`, o banco de dados será criado e não será apagado.
- d) `alter`, o banco de dados será atualizado e não será apagado.

11. (VUNESP/CM SOB/2023) No framework Hibernate do Java, o estado conhecido como `desanexado` (`detached`) ocorre quando

- a) o garbage collector da JVM detecta um objeto sem referência, mas ainda não o destruiu de fato.
- b) um objeto de uma classe persistente é associado a uma sessão e adquire uma representação no banco de dados, e também um valor de identificador.
- c) um objeto persistente acabou de ser destruído e sua representação no banco de dados foi excluída, sendo que a referência ao objeto não é mais válida.
- d) um objeto de uma classe persistente acabou de ser instanciado, mas ainda não está associado a uma sessão Hibernate.
- e) um objeto já esteve no estado persistente, mas teve sua sessão fechada, sendo que a referência ao objeto ainda é válida.

12. (CEBRASPE/AGER MT/2023) Assinale a opção que indica corretamente o exemplo de framework de persistência de dados em que há o mapeamento entre uma representação do modelo de objeto para uma representação do modelo de dados relacional.

- a) hibernate
- b) spring boot
- c) ESB (enterprise service bus)
- d) RMI (remote method invocation)
- e) MVC (model-view-controller)

13. (VUNESP/Pref. Pindamonhangaba/2023) No framework Hibernate do Java, o estado de um objeto de uma classe persistente que acabou de ser instanciado, mas que ainda não está associado a uma sessão, não possui representação no banco de dados e também não possui um identificador atribuído é conhecido como

- a) inicial.
- b) transiente.
- c) persistente.
- d) desanexado (detached).
- e) flutuante.

14. (FGV/ALEMA/2023) Analise o código a seguir, escrito em linguagem Java e com anotações do Hibernate 6.1. Considere que os métodos de acesso get e set foram omitidos da especificação das classes abaixo para efeitos de brevidade.

```
1 @MappedSuperclass
2 public static class Pessoa {
3
4     @Id
5     private Long id;
6
7     private String nome;
8
9     @Column(length = 11)
10    private String cpf;
11 }
12
13 @Entity(name = "Professor")
14 public static class Professor extends Pessoa {
15
16     private BigDecimal salario;
17 }
18
19 @Entity(name = "Aluno")
20 public static class Aluno extends Pessoa {
21
22     private boolean diurno;
23 }
```

Sobre o modelo de banco de dados decorrente deste mapeamento objeto-relacional, é correto afirmar que

- a) o atributo nome da tabela Pessoa será do tipo VARCHAR.
- b) o atributo salario da tabela Professor não poderá ser nulo.
- c) o atributo id da tabela Aluno será uma chave primária.
- d) o atributo diurno da tabela Pessoa poderá ter só 2 valores.
- e) o atributo cpf da tabela Pessoa poderá ter 11 caracteres.

15. (CEBRASPE/SERPRO/2023) Julgue o próximo item, relativo a tecnologias backend.

Em relação à gestão de bloqueio em um banco de dados relacional, o Hibernate 6.2 utiliza a estratégia otimista, uma vez que essa estratégia assume que várias transações podem ser concluídas sem afetar outras, logo, nessa versão, prescinde-se da estratégia pessimista.

16. (FGV/SEFAZ AM/2022) O Hibernate, versão 6, é um framework que oferece mecanismos para armazenamento e recuperação de objetos Java por meio de mapeamento objeto-relacional.

Em relação a interface Session, que oferece operações sobre instâncias de classes de entidade mapeadas, assinale a afirmativa correta.

- a) Uma instância transiente pode se tornar persistente chamando o método detach (Object object).
- b) Qualquer instância retornada pelo método get (String entityName, Object id) é persistente.
- c) Uma instância persistente pode se tornar transiente por meio do método merge (T object).
- d) Todas as instâncias persistentes são marcadas para remoção chamando o método flush().
- e) O estado original das instâncias gerenciadas é carregado na memória chamando o método reload().

17. (CEBRASPE/PC PB/2022) Em desenvolvimento web, é comum a utilização de classes, tecnologias ou mesmo ferramentas e frameworks, como aquela que utiliza sua própria linguagem de consulta, o que facilita a etapa de desenvolvimento quanto ao acesso a bancos de dados e SQL. Esse exemplo enfatiza uma das vantagens do

- a) asynchronous javascript and XML.
- b) servlet.
- c) hibernate.
- d) javaserver faces.
- e) javaserver pages.

18. (FCC/TRT 22/2022) Uma analista está desenvolvendo um sistema usando JPA e deseja configurar a utilização do framework Hibernate como provedor de persistência, por meio da instrução `<provider>org.hibernate.ejb.HibernatePersistence</provider>` que deve ser colocada no arquivo

- a) hibernate.xml
- b) persistence-unit.xml
- c) persistence.xml
- d) orm-database-config.xml
- e) server.xml

19. (CEBRSAPE/FUB/2022) Julgue o item a seguir, a respeito de projeto e desenvolvimento de sistemas.

Hibernate é um modelo computacional que faz parte de um processo avançado de gerenciamento de memória dos protocolos comumente utilizados em redes legadas.

20. (FCC/TRT 19/2022) No Hibernate uma entity bean deve ter uma chave primária, que é indicada por meio da anotação

- a) @PrimaryKey
- b) @PrymaryId
- c) @Id
- d) @SpecialKey
- e) @PK

Comentários:

21. (FCC/TRT 14/2022) Considere o método abaixo, presente em uma aplicação que utiliza Hibernate 4.3 original, executado em condições ideais.

```
private void createAndStoreEvent(Processo p) {  
    Session session = HibernateUtil.getSessionFactory().getCurrentSession();  
    session.beginTransaction();  
    ____I____ ;  
    session.getTransaction().commit();  
}
```

Para salvar o objeto p na tabela do banco de dados utiliza-se, na lacuna I, a instrução

- a) session.save(p)
- b) session.deploy(p)

- c) session.add(p)
- d) session.persite(p)
- e) session.send(p)

22. (FGV/CGU/2022) Considere o desenvolvimento de uma aplicação em um contexto de programação concorrente, em que é esperada a execução simultânea de múltiplas tarefas computacionais. O time utiliza a linguagem de programação JAVA e o framework ORM Hibernate no projeto. Para obter um objeto do tipo SessionFactory, o time decide utilizar o arquivo de configuração e as seguintes classes:

org.hibernate.cfg.Configuration,
org.hibernate.service.ServiceRegistry e
org.hibernate.service.ServiceRegistryBuilder.

No início do desenvolvimento da primeira funcionalidade com linhas de execução paralelas (Threads), o time discute se é seguro compartilhar o objeto do tipo SessionFactory.

Um dos desenvolvedores afirma que é seguro.

O time concluiu acertadamente que esse desenvolvedor está:

- a) errado, pois a classe SessionFactoryImpl implementa a interface Referenceable;
- b) errado, pois a classe SessionFactoryImpl não é thread-safe;
- c) errado, pois a classe SessionFactoryImplementor não é thread-safe;
- d) correto, pois o objeto do tipo SessionFactory é imutável e seu estado interno contém os metadados ORM;
- e) correto, pois a classe SessionFactoryImplementor implementa a interface ConcurrentHashMap.

23. (CC TJAC/TJ AC/2022) Hibernate é o framework para persistência de dados utilizado em projetos Java. Sendo uma das opções a implementar o conceito de mapeamento objeto-relacional (ORM). Avalie as asserções a seguir e a relação proposta entre elas

I - Com a utilização do Hibernate, não há necessidade de escrever SQL "puro", pois ele utiliza seu próprio código, chamado de HQL (Hibernate Query Language)

PORQUE

II - O Hibernate transforma os dados da estrutura lógica de um banco de dados em objetos relacionais.

A respeito dessas asserções, assinale a opção correta.

- a) As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.

24. (QUADRIX/PRODAM AM/2022) Com relação ao Hibernate, assinale a alternativa correta.

- a) O Hibernate contém diversas funcionalidades, como engenharia reversa, exportação de esquemas e anotações próprias para simplificar a utilização da API.
- b) O Hibernate é considerado um framework completo em razão de não necessitar adicionar nenhuma dependência para o seu correto funcionamento.
- c) A anotação `org.hibernate.annotations.ForeignKey` tem como função determinar o nome de uma chave primária.
- d) Uma das vantagens do Hibernate é o fato de não haver necessidade de definir identificadores para as entidades persistentes.
- e) O elemento mapping do arquivo `hibernate.cfg.xml` é usado para definir o usuário de conexão ao banco de dados.

25. (CEBRASPE/TJ PA/2020) O Hibernate tem suporte para mais de um tipo de herança. Na estratégia Tabela por Classe,

- a) várias associações unidimensionais são suportadas.
- b) há suporte às anotações para a geração de Id (IDENTITY).
- c) a classe deve ter anotações AUTO.
- d) a classe deve ser escrita sem anotações IDENTITY.
- e) as propriedades das superclasses e subclasses são mapeadas na mesma tabela.

Comentários:

Vamos analisar as alternativas.

- a) Errado. A alternativa menciona "associações unidimensionais", mas esse termo não está relacionado diretamente à estratégia Tabela por Classe. O Hibernate suporta associações bidirecionais e unidirecionais em qualquer estratégia de herança.
- b) Errado. A estratégia IDENTITY não é suportada na herança Tabela por Classe, pois o identificador precisa ser compartilhado entre as tabelas, e o IDENTITY está vinculado à geração de IDs individuais em tabelas específicas.
- c) Errado. Não há exigência de uma anotação AUTO específica para essa estratégia. A herança é configurada com `@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)`.

- d) Certo. Na estratégia Tabela por Classe, as classes não devem utilizar a estratégia IDENTITY, devendo adotar estratégias como SEQUENCE ou TABLE para garantir que os identificadores possam ser gerenciados de forma consistente.
- e) Errado. Essa descrição corresponde à estratégia SINGLE_TABLE, onde todas as propriedades das superclasses e subclasses são mapeadas na mesma tabela.

■

GABARITO

GABARITO



1. Letra A
2. Letra B
3. Letra E
4. Certo
5. Errado
6. Letra A
7. Letra C
8. Letra D
9. Letra C

10. Letra A
11. Letra E
12. Letra A
13. Letra B
14. Letra C
15. Errado
16. Letra B
17. Letra C
18. Letra C

19. Errado
20. Letra C
21. Letra A
22. Letra D
23. Letra A
24. Letra A
25. Letra D

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.